

The SharpPortico Manual

VERSION 1.5.0 · .NET 10

OpenAPI contracts, turned into protobuf messages, gRPC services, generated C# clients and a REST gateway — at build time, with no reflection.

- → Every option, every mapping rule, every diagnostic — with the code it produces
- → A deep dive into the mapping: scalars, shapes, names, presence, operations
- → Using the generated messages, servers, clients and the proxy gateway
- → Complete worked example: one specification from YAML to running client

SharpPortico is published as SharpPortico.SourceGenerator, SharpPortico.Runtime and SharpPortico.Tools on nuget.org.
This manual describes version 1.5.0 exactly. Code shown as generated output is copied from a build of this version.

Contents

Preface 7
- Who this is for 7
- How the book is organised 7
- Conventions 7
- The three claims 8
- Reading paths 8

1. What SharpPortico Is 9
- The problem it removes 9
- The pipeline 9
- What a build produces 10
- What it deliberately does not do 10
- The three claims 10
- Where it fits 11

2. The Five-Minute Path 12
- 1. The project 12
- 2. The contract 12
- 3. The name 13
- 4. Build 14
- 5. The implementation 14
- 6. The host 15
- 7. If something is missing 15

3. What the Build Writes 17
- Generated source is in memory by default 17
- The two files 17
- Getting a real .proto file 17
- A generated file, in order 18
- What is not written 18
- Which switch removes what 19
- Why the build is reproducible 19

4. Declaring a Contract 20
- The content always comes from AdditionalFiles 20
- Route 1: MSBuild properties 20
- Route 2: the assembly attribute 20
- Using both, and what wins 21
- The three derived names 21
- Several specifications in one project 22
- A declaration checklist 22

5. Options Reference 23
- Emission 23
- Streaming and pagination 23
- Mapping and errors 23
- Authentication 24
- Naming 24
- Proxy 24
- One attribute, many options 25
- SharpPorticoOptions is a mirror, not a second input 25
- Defaults as a work item 25
- What to check before adding an option 26

6. OpenAPI 3.0 and 3.1 27

Why a rewrite rather than a second parser 27
The fleet example 27
Construct by construct 29
What "mapped with a loss" means 29
What is not rewritten 29

7. Scalars and Formats 31
The table 31
What the edge document shows 31
Large integers and unsigned values 32
Strings that are really bytes 32
Defaults and presence 32
Where the type comes from when a schema is not inline 33
Choosing a format in the document 33

8. Names 34
Two names for one field 34
Operations 34
Messages 34
Enumerations 35
Collisions 36
Names the generator will not change 36

9. Object Shapes and Composition 37
The ordinary case 37
allOf: merge 37
oneOf and anyOf: flatten with a discriminator 38
Free-form objects: Struct 38
Any JSON value: Value 38
A component that is not an object 39
What determines the shape 39

10. Arrays and Dictionaries 40
The element decides 40
Arrays of inline objects 40
Arrays at the edges of an operation 41
The two nullability losses 42
What is not generated 42

11. Enumerations 43
The mapping 43
Values and numbers 44
Sharing and collisions 44
Nullability 44
In the gateway 45
What is not generated 45

12. Nullability and Presence 46
The difference in one table 46
What becomes optional 46
The generated members 47
Wire and size 47
On the server 47
On the client 48
In the gateway 48
OpenAPI 3.1 48
When presence cannot be represented 49
Why not wrappers? 49

13. Operations and RPC Kinds 50

One operation, one RPC 50
Naming 50
The four kinds 50
The large-body heuristic 51
Pagination changes the kind 51
Request and response shape 51
Streaming on the client 52
What an operation loses 52

14. Parameters, Bodies and Pagination 53
Parameters become fields 53
The request body 53
No parameters and no body 54
Pagination detection 54
Why detection is name-based 55
Order and numbers 55
What HTTP adds that protobuf cannot 55

15. JSON and Media Types 56
Choosing a media type 56
What a JSON schema becomes 56
JSON-specific constructs that are ignored 56
JSON in the gateway 57
Binary over gRPC 57

16. Binary Bodies and Files 58
What becomes bytes 58
A binary response is a message 58
Uploads 58
Proxy mode and binary payloads 59
When to use base64 instead 59

17. What Does Not Fit 60
Document-level constructs 60
Operation-level constructs 60
Schema-level constructs 61
The losses that change behaviour 61
What the 3.1 rewrite reports 62
A worked example: sink.yaml 62
How to decide whether a loss matters 62

18. Reading the Generated Message Code 63
One class, no base class 63
Field shapes 63
Presence, in code 64
The repeated field 64
The wire methods 64
Equality, cloning and text 65
The descriptor is null 65
Extending a message 65

19. The Server Surface 67
The contract class 67
The base class 67
Errors and the call context 68
Hosting on ASP.NET Core 68
Threading and lifetime 69
What the server surface does not include 69

20. The Client Surface 70

The low-level client 70
The convenience client 70
The channel is yours 71
Failures 72
Interceptors 72
What the client does not do 72

21. Authentication Helpers 73
What is read 73
The metadata helpers 73
The interceptor 73
The limits, stated plainly 74

22. The Gateway: gRPC in Front of REST 75
Enabling it 75
One call, step by step 75
What gets sent 76
What comes back 76
The cache 77
What the gateway cannot do 77

23. JSON at the Gateway 78
Writing a request body 78
Reading a response 78
Wrapper unwrapping 79
What the helper cannot represent 79
The string-escaping defect 80

24. The Command Line 81
Usage 81
A text run 81
A JSON run 82
Exit codes and streams 83
What the command cannot do 83
Using it in CI 83

25. Diagnostics 84
Two channels, one message 84
Severity means one thing 84
The ids 84
Parse: SP1000 85
Parse: SP1002 and SP1003 85
Mapping warnings: SP2000 85
Mapping errors: SP2005–SP2008 85
Ids that 1.5.0 does not raise 86
Gating a build 87

26. Symptom Index 88
Build symptoms 88
Runtime symptoms 88
Two symptoms that look like generator bugs 89
A triage order 89
What to put in a report 89
When the answer is "that is the mapping" 90

27. The Generated Surface, Member by Member 91
Messages and enums 91
The contract class 91
The server base 91
The convenience client 92

Dependency injection 92
Authentication 92
The proxy 93
Options that change the surface 93

28. A Worked Example End to End 94
The document 94
What each rule does 94
The descriptor 95
The implementation and the call 97
What this example does not exercise 97

29. Glossary 98

Page numbers are the physical pages of this PDF; the cover is page 1. Every chapter starts on a new page. The running footer is added after printing.

Preface

This is the complete manual for SharpPortico 1.5.0: a C# source generator that reads an OpenAPI 3.0 or 3.1 document at build time and writes protobuf messages, a gRPC service, a C# client, and — when you ask for it — a gateway that serves the same gRPC contract by calling the REST service you already have.

There are shorter documents in the repository: a README that fits on one page, a developer guide for adoption, a reference for lookup. This one is meant to be read. It explains not only *what* the generator emits, but why each rule exists, what the alternative would have cost, and what happens to a contract that does not fit the model. If you need to defend a mapping decision in a design review, the argument is in here.

Who this is for

You will get the most out of this book if one of these is your situation.

- **You are adopting SharpPortico.** Part I is a five-minute path from a package reference to a running client, and it tells you exactly which files the build produces so nothing appears by surprise.
- **You own an OpenAPI document** that other teams consume and you need to know what a given construct becomes on the wire — before you commit to it. Parts II and III are the contract.
- **You are writing the service or the client.** Part IV covers the generated surface: messages, servers, clients, authentication.
- **You are putting gRPC in front of a legacy REST API.** Part V is proxy mode, including its JSON mapping, its cache, its key handling and its limits.
- **A build is failing, or a mapping lost something.** Part VII lists every diagnostic and every symptom I know about, with the fix.

How the book is organised

Part	Chapters	What it establishes
I — Foundations	1–3	what the tool does, the five-minute path, what the build writes
II — Contracts	4–6	how a specification is declared, every option, the 3.0/3.1 split
III — The mapping deep dive	**7–17**	**the rules, stage by stage: scalars, names, shapes, arrays, enums, presence, operations, JSON, binary, losses**
IV — Generated code	18–21	messages, servers, clients, authentication in practice
V — Gateway	22–23	proxy mode end to end, and the JSON mapping it performs
VI — Command line	24	`sharpportico generate`, its report, its exit codes
VII — Diagnostics	25–26	every `SP` id, every symptom, and how to gate a build
VIII — Reference	27–29	the generated surface member by member, a worked example, a glossary

Part III is the heart of the manual and can be read on its own. Each chapter there states the rule, shows the input that exercises it, shows the output it produces, and names the loss when there is one.

Conventions

Code that is presented as output is output. Where a block says *generated*, it was produced by building this version of the generator against the input shown — not typed by hand to illustrate an idea. The `.proto` blocks in this book are the text a consumer's `{Service}Proto.Text` constant holds after a build, and CI compiles those same descriptors with `protoc`.

The fixtures are part of the book. `docs/manual/fixtures` holds the documents quoted in the mapping chapters — `sink.yaml`, `edge.yaml`, `fleet-3.1.yaml` and `petstore.yaml` — together with the descriptors 1.5.0 produced from them.

A rule in Part III can therefore be checked by reading both sides of it, and a snippet can be regenerated with `sharpportico generate` without reconstructing the input.

Options are named as they are declared — `EnableProxyGeneration`, `EmitGoogleRpcStatusWrapper` — and each one is given with its default. An option that the generator does not read is described as doing nothing, because a manual that implies a switch works is worse than one that admits it does not.

`SP` **ids are quoted exactly** as the compiler and the CLI print them, so they can be searched in a build log without guessing the punctuation.

One version. Everything here describes 1.5.0, the version in `Directory.Build.props` and on nuget.org. Nothing is described "as of 1.4" or "coming soon"; where behaviour changed, the change is named and the old behaviour is described only to explain an upgrade.

The three claims

SharpPortico makes three promises, and each one is checked by a job in CI rather than by a sentence in a document. They are repeated here because the rest of the manual assumes them.

Claim	How it is verified
Reflection-free, and NativeAOT-safe	`samples/NativeAotExample` is published as a native image, the job fails if a managed assembly is left behind, and the published binary runs to `SMOKE TEST PASSED`
The descriptor is real protobuf	every sample's `.proto` is compiled by `protoc` (`.github/scripts/check-protos.sh`)
The package works as installed	CI packs the `.nupkg`, consumes it from a throwaway project, and runs the packed tool on both target frameworks

The consequence for you is a specific kind of freedom: the generated messages are hand-written C# rather than classes produced by `protoc` and reflection, so they work where reflection does not — trimmed containers, single-file publishes, ahead-of-time compiled services.

Reading paths

- **In a hurry:** chapter 2, then chapter 19 (servers) or 20 (clients).
- **Reviewing a contract:** chapter 7, then the chapter in Part III that matches each construct you use.
- **Deciding whether to put gRPC in front of a legacy API:** chapters 22 and 23, in that order.
- **Explaining a warning to a colleague:** chapter 25, which has an entry per id.

1. What SharpPortico Is

SharpPortico is a Roslyn incremental source generator that reads an OpenAPI 3.0 or 3.1 document while your project compiles and writes a complete gRPC surface for it: protobuf message classes, a service definition, a server base class, a typed client, authentication helpers and — when enabled — a gateway that serves the gRPC contract by calling the REST service the document describes.

The input is a document you already maintain. The output is ordinary C# in your compilation: no `.proto` file has to be generated and compiled by `protoc`, no plugin runs outside MSBuild, and nothing is reflected over at run time. A build with the generator referenced behaves exactly like a build without it, except that several thousand lines of contract code appear in it.

The problem it removes

A REST contract is a description of HTTP: paths, verbs, query parameters, status codes, JSON bodies. A gRPC consumer needs something else: messages with numbered fields, one method per operation, a channel, and a client. Converting one to the other by hand has three predictable failure modes.

- **Drift.** The document changes and the hand-written `.proto` does not. The drift is invisible until a field arrives with the wrong number or an operation has no method.
- **Judgement calls made twice.** Is `format: int64` a `long` or a string? Is `nullable: true` still nullable on the wire? Every team answers these differently unless the answer is written down in code.
- **A toolchain to operate.** `protoc`, a gRPC plugin, a code generator per language, and a build step that has to run before the C# compiler. Each is another thing that can be missing from a clean checkout.

SharpPortico answers all three by making the mapping part of the compiler. The generator runs where the C# compiler runs, with the same inputs and the same cancellation, and emits the same files every time for the same document and version. The mapping rules are not configuration: they are code in `src/SharpPortico.SourceGenerator/Mapping`, and Part III of this manual is their specification.

The pipeline

TEXT

```
openapi/petstore.yaml
        │  AdditionalFiles item, or [assembly: OpenApiToGrpc(...)]
        ▼
OpenApi31Downgrade ── rewrite 3.1 constructs a 3.0 reader cannot parse
        ▼
OpenApiParser ──────── paths → RPCs, schemas → messages and enums
        │                (SchemaMapper owns types, shapes and presence)
        ▼
GrpcModel ──────────── an immutable intermediate representation
        │
        ├── ProtoEmitter   → {Service}Proto.Text (a proto3 descriptor as a C# const)
        ├── MessageEmitter → protobuf messages implementing IMessage<T>
        ├── ServiceEmitter → service definition, server base, typed client, DI, auth
        └── ProxyEmitter   → {Service}Proxy : {Service}Base (optional)
        ▼
obj/.../generated/.../{spec}.g.cs and {spec}.proto.cs
```

The intermediate model matters to you for one reason: every emitter reads the same model, so the descriptor, the C# messages and the gateway cannot disagree about a field's type or number. The CLI runs the same parser and the same `ProtoEmitter`, which is why `sharpportico generate` can print the descriptor before anything is built.

What a build produces

For a specification named `openapi/petstore.yaml`, with the service name `PetService` and the namespace `MyApp.Generated`:

Generated type	Kind	Purpose
`Pet`, `ListPetsRequest`, `ListPetsResponse`, ...	`sealed partial class`	protobuf messages, hand-written and reflection-free
`StatusEnum`, ...	`enum`	protobuf enumerations, emitted as C# enums
`PetService`	`static partial class`	method descriptors, `BindService`, marshallers
`PetService.PetServiceClient`	nested class	the `Grpc.Core.ClientBase` client
`PetServiceBase`	`abstract partial class`	overridable server methods with `Unimplemented` defaults
`PetServiceClient`	`sealed partial class`	C# 14 convenience client with typed overloads
`ServiceCollectionExtensions`	`static class`	`AddSharpPorticoPetService()`
`PetServiceAuth`, `PetServiceAuthInterceptor`	`static class`, `sealed class`	only when `securitySchemes` exist
`PetServiceProxy`	`sealed partial class`	only when `EnableProxyGeneration = true`
`PetServiceProto`	`static partial class`	`public const string Text`, the `.proto` descriptor

The full member-by-member list is chapter 27. What matters here is the shape of the result: the messages and the service are the contract, and everything else is convenience built on top of them.

What it deliberately does not do

A manual is only useful if it says where the tool stops.

- **It does not serve HTTP.** The generated code is a gRPC server and client. There is no ASP.NET Core endpoint, no JSON transcoding and no `google.api.http` annotation. Hosting is the job of `Grpc.Core` or `Grpc.AspNetCore`, as chapter 19 shows.
- **It does not read a `.proto` file.** The descriptor is an output, not an input. If you already have protobuf contracts, this is the wrong direction.
- **It does not generate OpenAPI.** The document is the source of truth; the generator never writes one.
- **It does not call `protoc`.** The message classes are emitted directly. `protoc` is used in this repository's CI to prove that the descriptor is valid, not to produce code.
- **It does not preserve every JSON Schema keyword.** Validation keywords such as `minLength`, `pattern`, `minimum` and `maxItems` are documentation. They do not become protobuf fields, because protobuf has no place to put them. Chapter 17 lists every such loss.
- **It does not make a legacy REST API into a gRPC API by itself.** Proxy mode does that at run time by translating each call back into HTTP; the legacy service stays the implementation and keeps its semantics, including its latency.

The three claims

The manual assumes three properties, and each one is checked by CI rather than asserted here.

Claim	What proves it
Reflection-free and NativeAOT-safe	`samples/NativeAotExample` is published with `PublishAot`, the published image must contain no managed assembly, and the binary must print `SMOKE TEST PASSED`
The descriptor is real protobuf	every sample's `{Service}Proto.Text` is written out and compiled by `protoc` in `.github/scripts/check-protos.sh`

Claim	What proves it
The package works as installed	CI packs the NuGet package, consumes it from a throwaway project, and runs the packed CLI on both target frameworks

Reflection-free is not a slogan about style. The generated messages implement `IMessage<T>` themselves: `CalculateSize`, `WriteTo`, `MergeFrom`, `Clone`, `Equals` and `GetHashCode` are written as code, and the gateway's JSON reader and writer are generated per message. That is what allows a trimmed or ahead-of-time-compiled service to run without a descriptor registry, and it is why the gateway does not use `System.Text.Json`'s reflection-based serializer.

Where it fits

SharpPortico is a good fit when:

- you have an OpenAPI document that is already the contract between teams, and
- you want gRPC consumers — internal services, mobile clients, an AOT service — without maintaining a second contract, and
- you can accept the mapping rules in Part III as the contract's protobuf meaning.

It is a poor fit when the document is generated from code you can change freely (change the code instead), when the API's semantics depend on HTTP details protobuf cannot carry (caching headers, content negotiation, `multipart/form-data`), or when you need bidirectional compatibility with an existing `.proto` that was written by hand — in that case the existing `.proto` is the contract, and a mapping tool would only be a second opinion.

The next chapter builds the smallest working service, so that the rest of the manual has something concrete to refer to.

2. The Five-Minute Path

This chapter takes an empty project and ends with a gRPC server answering a generated client, using the petstore contract that ships in `samples/`. Every file shown is a file the sample actually has.

1. The project

The generator is a NuGet package and a compiler extension, so a normal SDK project is enough. `net10.0` is used here; the generator itself targets `netstandard2.0` and loads into the compiler on either supported runtime.

MSBUILD / XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="SharpPortico.SourceGenerator" Version="1.5.0" PrivateAssets="all" />
    <PackageReference Include="Google.Protobuf" />
    <PackageReference Include="Grpc.Core" />
    <PackageReference Include="Grpc.Core.Api" />
    <PackageReference Include="Grpc.Net.Client" />
  </ItemGroup>

  <ItemGroup>
    <AdditionalFiles Include="openapi/**/*.yaml" />
  </ItemGroup>
</Project>
```

The `<AdditionalFiles>` item is not optional. It is where the generator reads the specification from, and it is the item type Roslyn provides for exactly this purpose. A file that is not an `AdditionalFiles` item is invisible to the generator, even when it sits next to the project.

2. The contract

`samples/MinimalApiExample/openapi/petstore.yaml` is deliberately small: three operations, one schema, one inline enumeration.

YAML

```
openapi: "3.0.3"
info:
  title: Petstore
  version: "1.0.0"
paths:
  /pets:
    get:
      operationId: listPets
      parameters:
        - name: limit
          in: query
          schema: { type: integer }
        - name: page
          in: query
          schema: { type: integer }
      responses:
        "200":
          description: paged pets
          content:
            application/json:
```

```
              schema:
                type: array
                items: { $ref: "#/components/schemas/Pet" }
    post:
      operationId: createPet
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Pet" }
      responses:
        "201":
          description: created pet
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Pet" }
  /pets/{petId}:
    get:
      operationId: getPet
      parameters:
        - name: petId
          in: path
          required: true
          schema: { type: integer, format: int64 }
      responses:
        "200":
          description: a pet
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Pet" }
components:
  schemas:
    Pet:
      type: object
      properties:
        id: { type: integer, format: int64 }
        name: { type: string }
        status:
          type: string
          enum: [available, pending, sold]
```

3. The name

With no configuration the service is named after the file (`petstore` → `PetstoreService`) and its namespace is that name. To name it explicitly, set two MSBuild properties; the package's `.props` file makes them visible to the compiler.

MSBUILD / XML

```
<PropertyGroup>
  <SharpPorticoServiceName>PetService</SharpPorticoServiceName>
  <SharpPorticoNamespace>MyApp.Generated</SharpPorticoNamespace>
</PropertyGroup>
```

Alternatively, when the project references `SharpPortico.Abstractions` (as the repository's samples do), an assembly attribute carries the same three values and every mapping option:

C#

```
using SharpPortico;

[assembly: OpenApiToGrpc(
    "openapi/petstore.yaml",
    "PetService",
    "SharpPortico.Samples.MinimalApi.Generated")]
```

The attribute is `Assembly`-scoped and `AllowMultiple = true`; several specifications can be declared in one project. The file path is resolved against the project directory, and the content still comes from the `AdditionalFiles` item — the

attribute customizes generation, it does not embed the document.

4. Build

```shell
dotnet build
```

Two files appear in the compiler's generated-output directory, and they are the whole story:

```text
obj/Debug/net10.0/generated/SharpPortico.SourceGenerator/
  SharpPortico.Generator.SharpPorticoGenerator/
    petstore.g.cs        messages, service, client, server base, DI
    petstore.proto.cs    PetServiceProto.Text, the descriptor as a const
```

The directory name is Roslyn's, not SharpPortico's: `<generator assembly>/<generator type>`. Chapter 3 explains exactly what each file contains and how to get a `.proto` file out of the build.

5. The implementation

The server side is one class. Only the methods you care about are overridden; the rest inherit the generated `Unimplemented` behaviour.

```C#
using Grpc.Core;
using SharpPortico.Samples.MinimalApi.Generated;

internal sealed class PetServiceImpl : PetServiceBase
{
    private readonly Dictionary<long, Pet> _pets = new()
    {
        [1] = new Pet { Id = 1, Name = "Rex", Status = StatusEnum.Available },
        [2] = new Pet { Id = 2, Name = "Milo", Status = StatusEnum.Pending }
    };

    public override Task<GetPetResponse> GetPetAsync(GetPetRequest request, ServerCallContext context)
    {
        if (!_pets.TryGetValue(request.PetId, out var pet))
            throw new RpcException(new Status(StatusCode.NotFound, $"pet {request.PetId} not found"));
        return Task.FromResult(new GetPetResponse { Data = pet });
    }

    public override Task<CreatePetResponse> CreatePetAsync(CreatePetRequest request, ServerCallContext context)
    {
        if (request.Body is null) throw new RpcException(new Status(StatusCode.InvalidArgument, "body
required"));
        _pets[request.Body.Id] = request.Body;
        return Task.FromResult(new CreatePetResponse { Data = request.Body });
    }

    public override Task<ListPetsResponse> ListPetsAsync(ListPetsRequest request, ServerCallContext context)
    {
        var page = request.Page <= 0 ? 1 : request.Page;
        var limit = request.Limit <= 0 ? 20 : request.Limit;
        var response = new ListPetsResponse();
        response.Items.Add(_pets.Values.OrderBy(p => p.Id).Skip((page - 1) * limit).Take(limit));
        return Task.FromResult(response);
    }
}
```

Two details are worth noticing now, because they are the mapping in miniature.

- `GetPetRequest.PetId` is a `long`, because the parameter declared `format: int64`. It is one member of the request message, and the path template it came from is recorded on the method descriptor.
- `ListPetsResponse.Items` is a `RepeatedField<Pet>`, and `CreatePetRequest.Body` is a `Pet`. Array responses become repeated fields on a wrapper; a `$ref` body becomes a message-typed field named `body`.

6. The host

`Grpc.Core.Server` is enough for a console host and keeps the example honest: the generated code has no hosting dependency of its own.

```C#
using Grpc.Core;
using Grpc.Net.Client;
using SharpPortico.Samples.MinimalApi.Generated;

const int Port = 50051;

var server = new Server
{
    Services = { PetService.BindService(new PetServiceImpl()) },
    Ports = { new ServerPort("localhost", Port, ServerCredentials.Insecure) }
};
server.Start();

using var channel = GrpcChannel.ForAddress($"http://localhost:{Port}");
var client = PetServiceClient.Create(channel);

var pet = await client.GetPetAsync(1);                 // scalar overload: long petId
Console.WriteLine($"{pet.Data?.Name} ({pet.Data?.Status})");

var created = await client.CreatePetAsync(new Pet { Id = 3, Name = "Luna", Status = StatusEnum.Sold });
Console.WriteLine($"created {created.Data?.Id}");

var page = await client.ListPetsAsync(new ListPetsRequest { Limit = 1, Page = 1 });
Console.WriteLine($"{page.Items.Count} pet(s)");

await server.ShutdownAsync();
```

`PetServiceClient.Create(channel)` is the convenience client. It is a `sealed partial class` deriving from the nested `PetService.PetServiceClient`, and it adds `Task<T>`-returning methods plus overloads that take the interesting parameters directly. The nested client from `Grpc.Core` remains available when you need `AsyncUnaryCall<T>`, `CallOptions` or per-call headers.

7. If something is missing

Symptom	Cause	Fix
no generated file at all	the YAML is not an `AdditionalFiles` item	add `<AdditionalFiles Include="openapi/**/*.yaml" />`
`SP1000` warning and no output	wrong path, empty file, or invalid YAML	check the path relative to the project, then the syntax
service named `PetstoreService` when you wanted `PetService`	no name configured	set `SharpPorticoServiceName`, or use the attribute
generated types are in an unexpected namespace	no namespace configured	set `SharpPorticoNamespace`
`CS0246` for `OpenApiToGrpc`	the attribute's assembly is not referenced	use the MSBuild properties, or reference `SharpPortico.Abstractions`
types appear twice in IntelliSense	the generator was referenced twice	reference the package once; do not also add `<Analyzer Include="...SharpPortico.SourceGenerator.dll" />`

Diagnostics are covered in chapters 25 and 26. The next chapter is the inventory of what the build wrote.

3. What the Build Writes

A generator that writes code into your compilation owes you an inventory. This chapter lists every file a SharpPortico build produces, where it lands, what is inside it, and which switch removes it.

Generated source is in memory by default

A Roslyn source generator adds syntax trees to the compilation; it does not have to write files. By default the trees exist only for the duration of the build, which is why a clean project can use the generated types in IntelliSense without a `Generated/` folder on disk.

To look at them, ask the compiler to persist them:

```shell
dotnet build -p:EmitCompilerGeneratedFiles=true -p:CompilerGeneratedFilesOutputPath=obj/generated
```

The path is then:

```text
obj/generated/SharpPortico.SourceGenerator/
  SharpPortico.Generator.SharpPorticoGenerator/
    petstore.g.cs
    petstore.proto.cs
```

Both segments are Roslyn's convention — the generator assembly name, then the generator type's full name — not names SharpPortico chooses. Set the property in a `.props` file or on the command line; committing the folder is not necessary and not recommended, because the files are rebuilt from the document on every build.

The two files

For `openapi/petstore.yaml`, every generated file's hint name comes from the specification's file name without its extension: `petstore`. The extension says what the file is.

File	Contains	Controlled by
`petstore.g.cs`	messages, enums, service definition, server base, client, DI, auth, proxy	the four `Emit*` switches
`petstore.proto.cs`	`PetServiceProto.Text`, a `const string` holding the proto3 descriptor	`EmitProtoFile` (default `true`)

There is no third file, and there is no `.proto` file on disk. The descriptor is emitted as a C# constant so that it is part of the compilation — trimmable, inspectable, and impossible to get out of step with the messages. `ProtoEmitter` produces the text; `CodeEmitter` escapes it into a string literal and names the class `{Service}Proto`.

```C#
namespace MyApp.Generated
{
    public static partial class PetServiceProto
    {
        /// <summary>The .proto descriptor for the generated service.</summary>
        public const string Text = "syntax = \"proto3\";\n\npackage myapp.generated;\n...";
    }
}
```

Getting a real `.proto` file

Two routes produce the same bytes.

At run time, write the constant out:

```C#
await File.WriteAllTextAsync("petstore.proto", PetServiceProto.Text);
```

Before a build, use the CLI. It runs the same parser and the same emitter, so the file is what the const will hold:

```SHELL
sharpportico generate openapi/petstore.yaml \
  --service-name PetService \
  --namespace MyApp.Generated \
  --out contracts/
```

The result is `contracts/petstore.proto` (the hint name, as `--out` names a directory) or exactly the path given when `--out` ends in `.proto`. Chapter 24 covers the command in full. CI in this repository does the second: it writes each sample's descriptor and compiles it with `protoc`, which is how the claim "the descriptor is real protobuf" is tested rather than asserted.

A generated file, in order

`petstore.g.cs` is one namespace block containing types in a fixed order. The order is stable, which makes diffs between two builds meaningful.

1. **Enums.** One `public enum` per OpenAPI enumeration, members in declaration order.
2. **Messages.** Component schemas first, then request and response wrappers in operation order, then inline nested messages as they are encountered.
3. **Service.** `static partial class PetService` with descriptors and `BindService`.
4. **Server base.** `abstract partial class PetServiceBase`.
5. **Convenience client.** `sealed partial class PetServiceClient`.
6. **DI.** `static class ServiceCollectionExtensions`.
7. **Auth.** `PetServiceAuth` and `PetServiceAuthInterceptor`, when the document declares security schemes.
8. **Proxy.** `PetServiceProxy`, when proxy generation is enabled.

Every generated class — messages, service, server base, client, DI, authentication and proxy — carries `[GeneratedCode("SharpPortico", "1.5.0")]`; enums do not. The version is read from the generator assembly at build time, so a stack trace or a committed file says which generator produced it. Do not edit generated code: the next build replaces it, and the replacement is not a merge.

What is *not* written

You might expect	Reality
a `.csproj` change per schema	none; the project is untouched
a `.proto` file compiled by `protoc`	no; the descriptor is a string, and the message classes are emitted directly
a `Generated/` folder by default	no; generated trees live in memory unless `EmitCompilerGeneratedFiles` is set
reflection metadata or a descriptor registry	no; `IMessage<T>` is implemented by hand, and the gateway's JSON code is generated per message
one file per message	no; one `.g.cs` per specification

Which switch removes what

Option	Default	Effect on the output
`EmitProtoFile`	`true`	`false` removes `{spec}.proto.cs`; the C# types are unaffected
`EmitClient`	`true`	`false` removes the convenience `{Service}Client`; the nested `{Service}.{Service}Client` from `Grpc.Core` remains. The DI extension still constructs the convenience client, so `EmitDependencyInjection` must be `false` too
`EmitServer`	`true`	`false` removes `{Service}Base`; the `BindService` overloads remain and reference it, so the generated file does not compile
`EmitDependencyInjection`	`true`	`false` removes `ServiceCollectionExtensions`

The four switches are read from the attribute. The `AdditionalFiles` route always emits all four, because that route has no attribute to carry the values; use the attribute when you need to turn one off.

Why the build is reproducible

The generator is incremental. Its inputs are the specification's text, the work item's options and the generator version; its output is a function of those alone. Nothing reads the clock, the machine name, the current directory or a random seed. Field numbers follow declaration order, message names follow the rules in chapter 8, and the descriptor is emitted with `\n` line endings regardless of the host.

That is also why the version is part of the output: two builds of the same document with different generator versions can legitimately differ, and the `[GeneratedCode]` attribute makes that difference visible instead of leaving a reviewer to guess.

The next part turns to the input side: how a contract is declared and which options actually change it.

4. Declaring a Contract

A specification reaches the generator through an `AdditionalFiles` item; its name, namespace and options come from MSBuild properties, per-file metadata, or an assembly attribute. This chapter describes all three routes, the precedence between them, and how the three names a build needs are derived when you configure nothing.

The content always comes from AdditionalFiles

The generator reads documents from Roslyn's `AdditionalTextsProvider`. There is no second way to supply the text: an attribute names a file, but the file must still be an `AdditionalFiles` item in the project. This is deliberate. `AdditionalFiles` is how a project tells the compiler "this is an input to an analyzer", it is incremental and cached by Roslyn, and it works identically in `dotnet build`, Visual Studio and CI.

MSBUILD / XML

```
<ItemGroup>
  <AdditionalFiles Include="openapi/**/*.yaml" />
  <AdditionalFiles Include="openapi/**/*.json" />
</ItemGroup>
```

The supported extensions are `.yaml`, `.yml` and `.json`, compared case-insensitively. Any other `AdditionalFiles` item is ignored by SharpPortico, so the same item group can carry files for other analyzers.

Route 1: MSBuild properties

The package ships a `.props` file that makes two properties and two per-file metadata names visible to the compiler:

MSBUILD / XML

```
<PropertyGroup>
  <SharpPorticoServiceName>PetService</SharpPorticoServiceName>
  <SharpPorticoNamespace>MyApp.Generated</SharpPorticoNamespace>
</PropertyGroup>
```

`SharpPorticoServiceName` is the base name; the suffix rule in the next section applies to it. The namespace is used verbatim for the generated C#, and the protobuf package is that namespace lower-cased.

The same values can be attached to one file, which is how several specifications in one project get different names:

MSBUILD / XML

```
<AdditionalFiles Include="openapi/pets.yaml"
                 SharpporticoServiceName="PetService"
                 SharpporticoNamespace="MyApp.Pets" />
<AdditionalFiles Include="openapi/orders.yaml"
                 SharpporticoServiceName="OrderService"
                 SharpporticoNamespace="MyApp.Orders" />
```

Note the spelling: the MSBuild properties are `SharpPorticoServiceName` (capital P), while the item metadata names are `SharpporticoServiceName` (lower-case p after the first word). The generator accepts exactly those names; a typo silently falls back to the file name.

Route 2: the assembly attribute

`SharpPortico.Abstractions` defines `OpenApiToGrpcAttribute`, which carries the file path, the service name, the namespace, the four `Emit*` switches and every mapping option:

C#

```
using SharpPortico;

[assembly: OpenApiToGrpc(
    "openapi/petstore.yaml",
    "PetService",
    "SharpPortico.Samples.MinimalApi.Generated",
    EnableProxyGeneration = true)]
```

The file path is resolved against the project directory. The attribute is `Assembly`-scoped and `AllowMultiple = true`, so one file can declare several specifications.

The attribute route requires a reference to the assembly that declares the attribute. In this repository the samples use a `ProjectReference` to `src/SharpPortico.Abstractions`; a consumer using the NuGet package can use the property route instead, which needs no compile-time reference at all.

Using both, and what wins

The two routes are merged. The attribute work items come first; an `AdditionalFiles` item whose **file name** matches an attribute's file name is dropped. The match is on the file name, not the full path, and it is case-insensitive.

That is why the samples can declare both and still generate once:

C#

```
[assembly: OpenApiToGrpc("openapi/petstore.yaml", "PetService", "SharpPortico.Samples.MinimalApi.Generated")]
```

MSBUILD / XML

```
<AdditionalFiles Include="openapi/**/*.yaml" />
```

The attribute wins, which is what allows it to configure options that the `AdditionalFiles` route cannot express. Two `AdditionalFiles` items with the same file name in different directories are **not** deduped against each other; they produce two work items and two generated files with the same hint name, which Roslyn rejects. Give such files distinct names, or declare them through attributes with distinct paths.

The three derived names

When nothing is configured, the generator derives a service name, a namespace and a protobuf package.

Name	Derivation	Example for `openapi/petstore.yaml`
service	file name without extension, made PascalCase, then `Service` appended unless already present	`PetstoreService`
namespace	the service name when `SharpPorticoNamespace` is absent	`PetstoreService`
package	the namespace, lower-cased	`petstoreservice`

The attribute route has one extra fallback: when the service name is omitted **and** the hint name is the literal `Service`, the first word of `info.title` is used as the stem. The `AdditionalFiles` route always has a file-stem hint, so it never reaches that fallback.

The suffix is configurable through `ServiceNameSuffix` (default `"Service"`), and the check is a plain `EndsWith`, so `PetService` stays `PetService` rather than becoming `PetServiceService`.

Several specifications in one project

Each work item produces its own `{hint}.g.cs` and `{hint}.proto.cs`, in its own namespace. There is no shared registry between them, so two contracts may declare a message with the same name as long as their namespaces differ.

The hint name is the file stem. Two specs named `openapi/v1/pets.yaml` and `openapi/v2/pets.yaml` both have the hint `pets`, and the second `AddSource` call fails the build. Rename one file, or give the specs distinct file names; the generated type names are unaffected by the file name once a service name is configured.

A declaration checklist

Question	Where the answer goes
Which document?	`AdditionalFiles` item, path relative to the project
What is the service called?	`SharpPorticoServiceName`, metadata, or the attribute's second argument
What namespace?	`SharpPorticoNamespace`, metadata, or the attribute's third argument
Which options?	attribute named arguments (chapter 5)
Several documents?	one item each, with distinct file names
JSON instead of YAML?	an `AdditionalFiles` item ending in `.json`; nothing else changes

The next chapter is the option reference: every property the attribute exposes, its default, and whether the generator reads it.

5. Options Reference

Every option below is a named argument on `[OpenApiToGrpc]`. The table gives its default and whether the generator reads it — because a switch that does nothing is worse than no switch at all, and this manual says which is which.

Emission

Option	Default	Read?	Effect
`EmitProtoFile`	`true`	yes	emits `{spec}.proto.cs`, the `{Service}Proto.Text` descriptor
`EmitClient`	`true`	yes	emits the convenience `{Service}Client`; the nested `Grpc.Core` client is always emitted. Turn `EmitDependencyInjection` off too when this is off, or the DI extension will not compile
`EmitServer`	`true`	yes	removes `{Service}Base`; the contract class still emits both `BindService` overloads, which take that type, so turning it off alone does not compile in 1.5.0
`EmitDependencyInjection`	`true`	yes	emits `ServiceCollectionExtensions.AddSharpPortico{Service}()`
`EmitGeneratedCodeAttribute`	`true`	**no**	every generated class always carries `[GeneratedCode("SharpPortico", version)]`; enums do not
`UseFileScopedNamespaces`	`true`	**no**	generated code always uses a block-scoped `namespace { … }`

The last two exist on the attribute and in `SharpPorticoOptions`, but no emitter consults them. Setting either to `false` changes nothing.

Streaming and pagination

Option	Default	Read?	Effect
`RespectStreamingHints`	`true`	yes	honours `x-grpc-streaming` and the large-body heuristic
`LargePayloadStreamingThresholdBytes`	`1_000_000`	yes	a `POST` body estimated at or above this size becomes client-streaming
`DetectPagination`	`true`	yes	recognises pagination parameters and may switch an RPC to server-streaming
`PaginationPageParameter`	`"page"`	yes	parameter name treated as the page number
`PaginationLimitParameter`	`"limit"`	yes	parameter name treated as the page size
`PaginationCursorParameter`	`"cursor"`	yes	parameter name treated as a cursor; implies server-streaming
`PaginationNextPageTokenParameter`	`"next_page_token"`	yes	parameter name treated as a next-page token; implies server-streaming

The parameter names are compared against the **proto** names of request fields, case-insensitively. A query parameter `pageSize` therefore does not match `limit`; set `PaginationLimitParameter` to `"page_size"` if your contract uses that spelling. Chapter 14 gives the full detection rules.

Mapping and errors

Option	Default	Read?	Effect
`MapEnumsToProtobufEnums`	`true`	**no**	enumerations are always protobuf enums; `false` does not turn them into C# enums in messages
`EmitDiagnosticsForUnmappableConstructs`	`true`	yes	reports `SP2000` for nullable arrays and nullable elements

Option	Default	Read?	Effect
`EmitGoogleRpcStatusWrapper`	`true`	yes	when an operation declares no successful response, emits a `{Method}Error` message with `code` and `message`
`Default4xxStatus`	`InvalidArgument`	**no**	unused; the gateway's HTTP-to-gRPC table is fixed (chapter 22)
`Default5xxStatus`	`Internal`	**no**	unused, for the same reason

`EmitGoogleRpcStatusWrapper` does not wrap a response that exists; it covers an operation with no `200`, `201`, `204` or `default` response at all. Turning it off leaves a response with only `has_value`.

Authentication

Option	Default	Read?	Effect
`GenerateAuthMetadataHelpers`	`true`	yes	emits `{Service}Auth` with a `Create…Metadata` method per security scheme
`GenerateAuthInterceptors`	`true`	yes	emits `{Service}AuthInterceptor` attaching the credential to every call shape

Both are conditional on the document declaring `components.securitySchemes`. With no schemes, neither type is generated regardless of the flags. The full behaviour is in chapter 21.

Naming

Option	Default	Read?	Effect
`ServiceNameSuffix`	`"Service"`	yes	appended to a derived service name when it does not already end with it
`MessageNaming`	`PascalCase`	**no**	message and field naming is fixed by the mapping rules in chapter 8
`MethodNaming`	`PascalCase`	**no**	RPC names always come from `operationId` through PascalCase sanitising
`StripApiVersionFromServiceName`	`true`	**no**	a `v1` / `v2` segment is never stripped; name the service explicitly to remove it

Proxy

Option	Default	Read?	Effect
`EnableProxyGeneration`	`false`	yes	emits `{Service}Proxy : {Service}Base`
`ProxyBaseUrl`	`servers[0].url`, else empty	**no**	resolved into the internal proxy model, but no emitter reads it; the address is supplied to `HttpRestClient` (chapter 22)
`ProxyApiKeyHeaderName`	`"X-Api-Key"`	**no**	accepted, but the generated proxy reads `ProxyOptions.ApiKeyHeaderName` at run time
`ProxyCacheTtlSeconds`	`60`	**no**	accepted, but the generated proxy reads `ProxyOptions.CacheTtl` at run time
`ProxyBypassCacheMetadataKey`	`"x-portico-bypass-cache"`	**no**	accepted, but the generated proxy reads `ProxyOptions.BypassCacheMetadataKey` at run time
`ProxyClientKeyHeaderName`	`"x-portico-key"`	**no**	accepted, but the generated proxy reads `ProxyOptions.ClientKeyHeaderName` at run time
`ProxyClientKeyMode`	`ClientKeyMode.None`	**no**	accepted, but the generated proxy reads `ProxyOptions.ClientKeyMode` at run time
`ProxyAuditEnabled`	`false`	**no**	accepted, but auditing is controlled by whether an `IProxyAuditLogger` is passed to the proxy constructor

Only `EnableProxyGeneration` changes what the build emits. The other proxy properties are carried into the internal model but not consulted by the emitter; the generated proxy reads the equivalent properties from the `ProxyOptions` instance it is

given at run time. Chapter 22 shows the object graph.

One attribute, many options

Named arguments are the only way to set these; the attribute constructor takes just the three strings.

```C#
[assembly: OpenApiToGrpc(
    "openapi/fleet.yaml",
    "FleetService",
    "MyApp.Fleet.Generated",
    RespectStreamingHints = true,
    LargePayloadStreamingThresholdBytes = 4_000_000,
    DetectPagination = true,
    PaginationLimitParameter = "page_size",
    EmitDiagnosticsForUnmappableConstructs = true,
    GenerateAuthMetadataHelpers = true,
    GenerateAuthInterceptors = true,
    EnableProxyGeneration = true)]
```

`ProxyClientKeyMode` is of type `SharpPortico.ClientKeyMode`, but it is not read either; the generated proxy uses `ProxyOptions.ClientKeyMode` at run time. The other enum-valued properties (`Default4xxStatus`, `Default5xxStatus`, `MessageNaming`, `MethodNaming`) are accepted by the compiler and are also not read, as noted above.

`SharpPorticoOptions` is a mirror, not a second input

`src/SharpPortico.Abstractions/SharpPorticoOptions.cs` declares a class with the same properties and the same defaults, plus `SharpPorticoOptions.Default`. It exists so code that manipulates options can do so as one object, because an attribute argument cannot be a class instance.

The generator does **not** read it. A project that constructs `SharpPorticoOptions` and expects the build to change is disappointed: only the attribute's named arguments reach the pipeline. The class carries a doc comment saying exactly this, and the generator's internal `OpenApiWorkItem` is the single source of truth for defaults.

Defaults as a work item

The effective configuration for a build is the `OpenApiWorkItem`. Its defaults are visible in `AttributeRequest.ToWorkItem()` and `AdditionalFileRequest.ToWorkItem()`:

Setting	Default in both routes
`RespectStreamingHints`	`true`
`LargePayloadStreamingThresholdBytes`	`1_000_000`
`EmitDiagnosticsForUnmappableConstructs`	`true`
`GenerateAuthMetadataHelpers`	`true`
`GenerateAuthInterceptors`	`true`
`DetectPagination`	`true`
page / limit / cursor / next token	`page` / `limit` / `cursor` / `next_page_token`
`EmitGoogleRpcStatusWrapper`	`true`
`ServiceNameSuffix`	`Service`
`EnableProxyGeneration`	`false`
`ProxyCacheTtlSeconds`	`60`
`ProxyClientKeyMode`	`None`

Setting	Default in both routes
`ProxyAuditEnabled`	`false`

The `AdditionalFiles` route has no way to override these; it exists to make a contract generate with sane defaults in a project that only wants the contract. Anything else is an attribute.

What to check before adding an option

1. **Is it in the "Read? yes" list?** If not, changing it is a no-op; the behaviour is fixed by the mapping.
2. **Does the document support it?** Auth options do nothing without `securitySchemes`; proxy options do nothing without `EnableProxyGeneration`.
3. **Is the name a proto name?** Pagination matching happens after field naming, so it compares `next_page_token`, not `nextPageToken`.
4. **Does the option change types?** Almost none do. The type of a field is the mapping's decision (Part III); options affect RPC kinds, emitted extras and the gateway, not scalar choices.

The next chapter is the one input distinction that does change types: whether the document declares OpenAPI 3.0 or 3.1.

6. OpenAPI 3.0 and 3.1

The bundled reader understands OpenAPI 3.0, not 3.1. SharpPortico therefore rewrites a 3.1 document's tree to its 3.0 equivalent before parsing it, maps every construct it can, and reports each construct whose meaning changed. Nothing about a 3.0 document changes.

Why a rewrite rather than a second parser

`Microsoft.OpenApi` 1.6.x refuses a document whose `openapi` field is `3.1.0`. Supporting 3.1 could mean a second parser for every mapping rule, or a translation step in front of the existing one. SharpPortico does the second: `OpenApi31Downgrade` loads the YAML tree, walks it, and emits the same document with 3.1-only constructs expressed in 3.0 form. The mapper then sees one dialect.

Two consequences follow, and both are visible to you.

- The rewrite is **structural**, not textual. It can tell a schema's `type` keyword from a property named `type`, and it handles YAML flow sequences, block sequences and JSON arrays identically.
- It is **reported**. `SP1002` says the document was parsed as 3.0 and lists the kinds of construct that were rewritten. `SP1003` names each construct whose mapping lost something.

A 3.0 document is returned byte for byte. A tree the rewrite cannot read falls back to rewriting only the declared version, so an unusual 3.1 file still gets a chance to parse rather than being rejected outright.

The fleet example

`samples/NativeAotExample/openapi/fleet-3.1.yaml` exists to exercise the rewrite. Its schema contains one of almost every construct that 3.0 cannot spell:

```yaml
openapi: 3.1.0
jsonSchemaDialect: https://json-schema.org/draft/2020-12/schema
components:
  schemas:
    Vehicle:
      type: object
      properties:
        id: { type: integer, format: int64 }
        callsign: { type: [string, "null"] }
        kind: { const: truck }
        telemetry: { type: [object, array] }
        bounds:
          type: array
          prefixItems:
            - { type: string }
            - { type: integer }
        manifest: { type: string, contentEncoding: base64 }
        modelYear: { type: integer, exclusiveMinimum: 0 }
        serviceWindow: { $ref: "#/$defs/ServiceWindow" }
        labels: { type: object, additionalProperties: true }
$defs:
  ServiceWindow:
    type: object
    properties:
      label: { type: string }
webhooks:
  vehicleRetired:
    post:
      operationId: onVehicleRetired
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Vehicle" }
```

The generated descriptor begins with the message that exercises the rewrite:

```proto
syntax = "proto3";

package fleet31service;

import "google/protobuf/struct.proto";

option csharp_namespace = "Fleet31Service";

enum KindEnum
{
    TRUCK = 0;
}

message Vehicle
{
    int64 id = 1;
    optional string callsign = 2;
    KindEnum kind = 3;
    google.protobuf.Value telemetry = 4;
    repeated google.protobuf.Value bounds = 5;
    bytes manifest = 6;
    int32 model_year = 7;
    ServiceWindow service_window = 8;
    google.protobuf.Struct labels = 9;
}
```

Every line of that message is explained by a rule in this chapter or the next part of the manual.

Construct by construct

3.1 construct	Rewritten to	Protobuf result	Loss reported
`openapi: 3.1.0`	`3.0.3`	—	`SP1002` (the notice)
`jsonSchemaDialect`	removed	—	`SP1003` : the dialect's semantics are not honoured
`type: [T, "null"]`	`type: T` , `nullable: true`	`optional T` on a singular scalar	none
`type: [A, B, …]` (several non-null)	`x-sharpportico-json-value: true`	`google.protobuf.Value`	`SP1003` : no protobuf union
`type: ["null"]` only	the same marker	`google.protobuf.Value`	`SP1003` : a value that is always null
`const: X`	`enum: [X]`	a single-member protobuf enum	none
`prefixItems` with one shared scalar type	`items: {type, format}`	`repeated` that scalar	`SP1003` : fixed length and positions are gone
`prefixItems` with differing types	`items:` the JSON-value marker	`repeated google.protobuf.Value`	`SP1003` : same
array with `items` and no `type`	`type: array`	`repeated`	none
`contentEncoding: base64`	`format: byte`	`bytes`	none
any other `contentEncoding`	removed	the underlying `string`	`SP1003` : encoding not applied
numeric `exclusiveMinimum: 0`	`minimum: 0` , `exclusiveMinimum: true`	the field's numeric type	none (bounds are documentation)
`$ref: "#/$defs/X"`	hoisted to `components/schemas/X` and repointed	a message named `X`	none
`webhooks`	merged under `/webhooks/{name}`	RPCs like any other operation	`SP1003` : a webhook is a call the API makes to *you*

What "mapped with a loss" means

A loss is not a refusal. The contract still generates, and the diagnostic says what the protobuf contract cannot express. For the fleet document, the build reports something like:

```
TEXT

warning SP1002: SharpPortico parsed 'openapi/fleet-3.1.yaml' as OpenAPI 3.0: ... it was rewritten to
the 3.0 equivalent first - the declared version, $defs references, webhooks, a type array, const,
prefixItems, contentEncoding, a numeric exclusive bound
warning SP1003: SharpPortico mapped the tuple at components/schemas/Vehicle/properties/bounds in
'openapi/fleet-3.1.yaml', but a JSON Schema tuple's fixed length and per-position types have no
protobuf equivalent, so it maps to a repeated field
warning SP1003: SharpPortico mapped webhooks in 'openapi/fleet-3.1.yaml', but 1 webhook operation(s)
were mapped to RPCs; a webhook is a call the API makes to you, so a client method for one describes
a payload and not a call the server accepts
```

The full text is in chapter 25. What matters here is the shape: one informational notice for the rewrite, then one warning per construct that did not survive intact. A build that treats warnings as errors will fail on a 3.1 document containing a tuple or a webhook — which is the intended way to make a team decide whether that loss is acceptable.

What is not rewritten

The rewrite handles constructs it knows. Keywords it does not recognise are left in the tree and then either read by the 3.0 reader or ignored. In particular:

- `unevaluatedProperties` and `unevaluatedItems` are not mapped to anything; `additionalProperties: true` still produces `google.protobuf.Struct`, as it does in 3.0.
- JSON Schema assertions (`pattern`, `minLength`, `maximum`, `contains`, `dependentRequired`) do not change a type and are not emitted as protobuf validation.
- `examples` and `example` are documentation; they do not appear in the descriptor.
- A `webhooks` entry whose generated path already exists under `paths` is left out, with an `SP1003` naming the collision.

If a 3.1 construct matters to your consumers, the test is simple: read the generated `.proto`. If the type and the field are what you meant, the loss was cosmetic; if not, the `SP1003` warning is the explanation and the contract should be changed to say what protobuf can carry.

Part III now turns to the mapping itself, beginning with the simplest question: what does a scalar become?

7. Scalars and Formats

A scalar is the smallest decision the mapping makes, and it is made in one place: `SchemaMapper.ResolveScalarKind`. This chapter states the table, then shows what the table does not do — because OpenAPI has more scalar formats than protobuf has scalar types.

The table

OpenAPI `type`	`format`	Protobuf	C#	Notes
`string`	absent, or anything but `binary` / `byte`	`string`	`string`	includes `uuid`, `date`, `date-time`, `email`, `uri`, `password`
`string`	`binary`	`bytes`	`ByteString`	file contents
`string`	`byte`	`bytes`	`ByteString`	base64 text
`integer`	absent	`int32`	`int`	
`integer`	`int64`	`int64`	`long`	
`integer`	any other format (`int8`, `int16`, `uint32`, …)	`int32`	`int`	the format is not consulted
`number`	`float`	`float`	`float`	
`number`	absent, `double`, or anything else	`double`	`double`	
`boolean`	—	`bool`	`bool`	
`file`	—	`bytes`	`ByteString`	OpenAPI 2.0 leftover, still read
absent or unrecognised	—	`string`	`string`	the fallback

The implementation is one `switch` over `type` and a `format` comparison per arm:

```C#
case "string":
    if (format is "binary" or "byte") return FieldKind.Bytes;
    return FieldKind.String;
case "integer":
    return format == "int64" ? FieldKind.Int64 : FieldKind.Int32;
case "number":
    return format == "float" ? FieldKind.Float : FieldKind.Double;
case "boolean":
    return FieldKind.Bool;
```

The rule is intentionally small. A mapper that tried to honour every format would need a type for each, and protobuf has no `uuid`, no `date`, no `uri` and no `decimal`. Choosing the nearest protobuf scalar and saying so is more useful than inventing wrapper messages nobody asked for.

What the edge document shows

`edge.yaml` declares an inline object whose properties use the formats that most often surprise people:

YAML

```yaml
GetThingByIdData:
  type: object
  properties:
    total: { type: integer }
    ratio: { type: number }
    when: { type: string, format: date-time }
    uid: { type: string, format: uuid }
```

The generated message is:

PROTO

```proto
message GetThingByIdData
{
    int32 total = 1;
    double ratio = 2;
    string when = 3;
    string uid = 4;
}
```

`date-time` and `uuid` are strings because that is what protobuf can carry without a convention. If a consumer needs a `Timestamp` or a `Guid`, the conversion belongs at the edge of that consumer, where the contract's textual format is known. The generator does not guess: `when` could be RFC 3339, a local time with an offset, or a date, and the schema does not say which.

Large integers and unsigned values

`integer` without `int64` is `int32`, even when the format says `uint32` or `int16`. Two consequences are worth stating plainly:

- A contract with `format: uint32` and values above `2_147_483_647` overflows the generated `int`. Declare `int64` in the document if the range matters, or accept the narrower field.
- Negative values are representable, which `uint32` would forbid. The mapping does not add validation; it picks the type that can hold the common case.

There is no path from an OpenAPI document to `uint32` or `uint64` in this version. The field kinds exist in the model and the emitters handle them, but `ResolveScalarKind` never returns them. A `.g.cs` you inspect will therefore not contain `uint` fields even when the document says `format: uint32`.

Strings that are really bytes

`format: binary` and `format: byte` both become `bytes`. The difference is only how the value is written in JSON:

Format	JSON representation	Generated field
`byte`	base64 string	`ByteString`
`binary`	raw octets, not JSON	`ByteString`

For an `application/octet-stream` body the mapper does not look at a schema at all: the field is always `bytes body`. Chapter 16 covers uploads and downloads.

Defaults and presence

A scalar field with no `nullable: true` has no presence. Its C# property returns the protobuf default — `0`, `false`, `""`, an empty `ByteString` — when nothing was sent, and `CalculateSize` writes nothing for a default value. This is proto3's design, not a SharpPortico choice.

A scalar with `nullable: true` becomes `optional`, gains `Has…` / `Clear…`, and a set default is written on the wire. Chapter 12 is the full treatment. The important interaction with this chapter is that **presence does not change the scalar type**: `optional int32` is still an `int`, and `optional string` is still a `string`.

Where the type comes from when a schema is not inline

A `$ref` is resolved to the referenced schema's type. If the reference names an object, the field is a message; if it names an enumeration, the field is that enum. The scalar table applies to the schema behind the reference, not to the reference's name. Chapter 9 covers references and composition; the rule here is simply that a scalar is always decided by `type` and `format`, never by a property's name or its `example`.

Choosing a format in the document

Because the mapping is small, the document has more control than it looks. Three habits prevent surprises.

1. **Use `int64` for identifiers and counts that can grow.** An `int32` ID is a limit the contract did not intend to declare.
2. **Use `format: byte` when JSON must carry binary data.** The gateway writes a `bytes` field as base64, so a `byte` field round-trips through JSON without a custom encoder.
3. **Do not use `format` to express semantics.** `uuid`, `date-time`, `email` and `uri` are conventions for consumers; they do not change the protobuf type, and the generated code does not validate them.

If a contract needs a real timestamp type, the protobuf contract is the place to say so: a component schema can declare a `google.protobuf.Timestamp` only if the source document can express it, and this mapping does not infer one from `format: date-time`. That is a deliberate refusal to guess, and it is listed among the losses in chapter 17.

8. Names

Names are where a mapping becomes visible in every line of consumer code. SharpPortico derives a C# identifier and a protobuf identifier from each OpenAPI name, and the two are not the same string. This chapter states both rules and the collisions they can produce.

Two names for one field

Every field carries a C# name and a proto name.

OpenAPI property	C# member	`.proto` field	Why
`petId`	`PetId`	`pet_id`	C# convention; proto snake_case
`x-rate-limit`	`XRateLimit`	`x_rate_limit`	separators are removed in C#, kept as `_` in proto
`URL`	`URL`	`u_r_l`	see the acronym rule below
`snake_case`	`SnakeCase`	`snake_case`	
`2fa`	`N2fa`	`_2fa`	identifiers may not begin with a digit
`status.code`	`StatusCode`	`status_code`	`.` is a separator

The C# rule (`SanitizePascal`) keeps letters and digits, drops every other character, and upper-cases the character that follows a separator. It does not lower-case the rest, so an acronym survives: `URL` stays `URL` .

The proto rule (`ToProtoName`) lower-cases, turns `_` , `-` , `.` , space and `:` into `_` , and inserts an underscore before an upper-case letter that follows a lower-case one. That last clause is why `petId` becomes `pet_id` ; it is also why an acronym is spelled out:

```text
URL     →  u _ r _ l
```

This is not an accident of the implementation; it is the predictable consequence of applying the same camel-case splitter to every input. If a contract needs `url` , write `url` .

Operations

An RPC is named from `operationId` , passed through `SanitizePascal` :

`operationId`	RPC
`listPets`	`ListPets`
`get-pet-by-id`	`GetPetById`
`Pets_Create`	`PetsCreate`
absent	`Op0POST` (operation index and verb)

The fallback counts operations in document order, so a document without `operationId` still gets stable, unique names — but the names say nothing about intent. Always declare `operationId` in a contract you intend to keep.

Messages

The wrappers around an operation are named after the RPC:

Message	Content
`{Method}Request`	parameters and body of the operation
`{Method}Response`	the successful response's fields
`{Method}Body`	an inline request body that is not a `$ref`
`{Method}Data`	an inline response object that is not a `$ref`
`{Method}Error`	the error wrapper when the operation declares no success response
`{Method}Item`	the element message of a top-level array response with inline items

Component schemas keep their component names: `components/schemas/Pet` becomes `message Pet`, with the name passed through `SanitizePascal` (`pet_summary` → `PetSummary`).

Inline objects are named after the property that declares them: a property `dimensions` with an inline object would be `Dimensions`, and a repeated property `children` would be `Children`. In 1.5.0 those nested messages are named but not emitted, so the field refers to a type the file does not contain; declare the object as a named component and use `$ref` (chapter 17). A property that already uses `$ref` is unaffected.

Enumerations

Where the enum appears	Generated name
`components/schemas/WidgetKind` with `enum`	`WidgetKind`
inline `enum` on property `state`	`StateEnum`
inline `enum` on repeated property `tags`	`TagsEnum`
3.1 `const: truck` on property `kind`	`KindEnum`
inline `enum` on top-level array response items	`{Method}ItemsEnum`

Members are `SanitizePascal` of the declared value. String members become PascalCase (`available` → `Available`); numeric members become `N` plus the number (`1` → `N1`, `10` → `N10`), and the number is kept as the protobuf value. In the `.proto`, `ToProtoEnumName` inserts an underscore before every upper-case letter except the first, so `InProgress` becomes `IN_PROGRESS` and `N1` stays `N1`.

PROTO

```
enum Level
{
    N1 = 1;
    N2 = 2;
    N3 = 3;
}

enum Priority
{
    N0 = 0;
    N10 = 10;
    N20 = 20;
}
```

The C# enum members use the PascalCase names (`Level.N1`), while the proto uses the upper-case form. A numeric enum whose first member is not zero produces a descriptor `protoc` rejects, because proto3 requires the first value of an enum to be `0`. If your numeric enumeration starts at 1, add a zero member in the document or accept that the C# types work while the descriptor does not compile.

Collisions

A name is not a suggestion: two types with one name in one namespace cannot compile, and a member may not have its type's name. The mapper checks four collisions and either derives a distinct name or refuses the document.

Id	Collision	Behaviour
`SP2005`	a component schema is named like an operation wrapper (`CreatePetRequest` , `CreatePetResponse` , …)	**error**, no output
`SP2006`	a property would generate a member with the enclosing type's name (`Pet.Pet`)	**error**, no output
`SP2007`	two inline enumerations share a name and declare different members, and the derived name also collides	**error**, no output
`SP2008`	a nullable property needs `Has{Name}` / `Clear{Name}` and the schema already declares that member	**error**, no output

Two inline enumerations with one name and **the same** members are one type and are shared, which is what a vocabulary reused across schemas should do. Two with different members are given a derived name made of the original name plus the declared members in order — `State` with `purged` becomes `StatePurged` . If that derived name is free, both enumerations survive without a diagnostic and without a name you wrote. The `SP2007` error is reserved for the case where even the derived name is taken by a different member set.

An operation wrapper collision is the most common in practice, because the wrapper names are derived from `operationId` rather than declared. A schema called `GetPetResponse` and an operation `getPet` cannot coexist:

```
TEXT

error SP2005: SharpPortico cannot map 'openapi/pets.yaml': the message name 'GetPetResponse' is produced
twice — an operation derives its messages as '{OperationId}Request' and '{OperationId}Response', so a
component schema of that name collides. Rename one of the two
```

Renaming the schema is usually right. Renaming the operation changes the RPC, the client method and every call site, which is a larger contract change than renaming a component nobody outside the document sees.

Names the generator will not change

Two identifiers are fixed:

- **`has_value`** : an operation or schema with no fields gets a single `bool has_value = 1` so that the message is not empty. It is spelled in proto and `HasValue` in C#. A contract that declares its own `has_value` property in that position produces duplicate members; rename the property.
- **`body`** , **`data`** , **`items`** : the request body, response payload and array payload fields. A parameter called `body` and a request body both map to `Body` , and the second field is not renamed; field-number assignment makes the collision a compile error rather than a silent overwrite.

The general advice follows from the rules: keep component names nouns, keep `operationId` verbs, and avoid property names that differ only in punctuation. The mapping will handle the rest deterministically.

9. Object Shapes and Composition

An object schema can be written in several ways: properties, `allOf`, `oneOf`, `anyOf`, a reference, or nothing at all. Protobuf has one construct for all of them — a message — so the mapper's job is to decide which fields that message has.

The ordinary case

An object with `properties` becomes a message with one field per property, in declaration order, numbered from 1. Nested objects are meant to become nested messages named after their property; in 1.5.0 an inline object under a property is not emitted, so declare that object as a named component and reference it with `$ref` (chapter 17). An inline response or request-body object is emitted as the wrapper's `data` or `body` message, so it does not need a component. Arrays become repeated fields (chapter 10); references become message- or enum-typed fields (chapter 11).

`sink.proto` shows the result for a schema that uses nesting and recursion:

PROTO

```
message Dimensions
{
    double width = 1;
    double height = 2;
}

message Widget
{
    // ...
    Widget child = 23;
    repeated Widget children = 24;
    Dimensions dims = 25;
    StateEnum state = 26;
}
```

`child` is a `Widget` because the property references the schema being defined. A self-reference is legal protobuf — a message may hold a field of its own type — so no cycle diagnostic is raised and no special case is needed. `dims` is a `Dimensions` message because the property is a `$ref` to the `Dimensions` component; the component is emitted because it has a name the model can carry.

allOf: merge

`allOf` is read as "all of these together", so the mapper merges the members:

1. each `allOf` entry is merged in order (its properties, then its own nested `allOf`);
2. the schema's own `properties` are merged last.

Field numbers follow the merged order. A property declared in two entries appears twice, because the mapper does not deduplicate: protobuf has no notion of overriding a field, and silently keeping one of two declarations would hide a contradiction in the contract. If your document uses `allOf` for inheritance, ensure the base and the extension do not declare the same property.

One case is special. A component whose entire body is a single `$ref`:

YAML

```
components:
  schemas:
    PetAlias:
      allOf:
        - $ref: "#/components/schemas/Pet"
```

is emitted as an **empty message**, not as an alias:

```proto
message PetAlias
{
}
```

proto3 has no type alias. The mapper could copy `Pet`'s fields into `PetAlias`, but then the two types would drift the moment `Pet` changed, and a consumer would have two incompatible messages that look identical. It chooses a distinct, empty type instead. The practical consequence is that a `$ref` to `PetAlias` gives a field with no members; reference `Pet` directly when the alias exists only to rename a type.

oneOf and anyOf: flatten with a discriminator

A union cannot be a protobuf field: a field has one type. The mapper's approximation is:

- if the union has exactly **one** choice, merge that choice's fields (the union adds nothing);
- otherwise add a `string kind = 1` discriminator field, then merge the fields of the **first choice that declares properties**, or the first choice when none do.

The merged fields are flat: a `oneOf` of `Cat` and `Dog` produces one message with `kind` plus the union of the two shapes' members, with the second shape contributing nothing if only the first was chosen. This is a loss, and it is a deliberate one: the alternative — one message per alternative plus a wrapper — would change the contract's type graph and force every consumer to write a switch that the document did not ask for. Chapter 17 records it among the constructs that do not survive intact.

`anyOf` is treated exactly like `oneOf`. JSON Schema draws a distinction (at least one versus exactly one); protobuf has no place to enforce either.

Free-form objects: Struct

An object that declares no members — `type: object` with no `properties` — says "any JSON object". Protobuf has a type for exactly that: `google.protobuf.Struct`, a map of string keys to `google.protobuf.Value`. A bare `{}` schema is different: it has no type at all, so it falls through to the scalar fallback and becomes a `string` (chapter 7).

```proto
message Widget
{
    // ...
    google.protobuf.Struct labels = 16;
    google.protobuf.Struct meta = 17;
}
```

`sink.yaml` declared `labels` as `type: object, additionalProperties: true` and `meta` as an object with no properties; both became `Struct`. The `google/protobuf/struct.proto` import is added automatically when the model contains one.

A schema whose `additionalProperties` names a **type** is still a free-form object in this mapping. There is no `map<string, T>` generation: the mapper checks for declared properties, not for a typed value schema, so `additionalProperties: { type: integer }` becomes `Struct` as well. That is a loss worth knowing before you write a contract that depends on a typed dictionary (chapter 17).

Any JSON value: Value

`google.protobuf.Value` is protobuf's "any JSON value": null, number, string, bool, list or object. The mapper uses it when:

- the schema carries the internal marker `x-sharpportico-json-value: true` (written by the 3.1 rewrite for a multi-type union or a heterogeneous tuple), or

- the 3.1 construct is a type union with more than one non-null type.

```proto
message Vehicle
{
    google.protobuf.Value telemetry = 4;
    repeated google.protobuf.Value bounds = 5;
}
```

`telemetry: { type: [object, array] }` has no protobuf type that is both an object and an array, so `Value` is the honest answer. `bounds` is a tuple whose positions differ in type; a repeated field holds one type, so each element is a `Value`.

`Value` and `Struct` are different answers to different questions. `Struct` is "an object whose members I do not describe"; `Value` is "a value whose type I do not describe". A contract that means one and gets the other loses shape or flexibility, so the mapper asks the question at the position (property, array element, body, response) rather than guessing from the component's name.

A component that is not an object

A component schema can be an array, a scalar, or empty. Each has a defined shape when it becomes a message.

Component schema	Generated message
`type: object` with properties	one field per property
`type: array`	`repeated <element> values = 1`
a scalar	`<scalar> value = 1`
`{}` or `type: object` with no properties, used as a component	`bool has_value = 1`
`allOf: [ $ref ]`	empty message (see above)

The `Values` / `Value` wrappers exist because a component has to be a message to be referenced by a field. When the same schema is used as a **property**, the mapper does not wrap it: a property whose schema is an array is a repeated field directly, and a property whose schema is a free-form object is `Struct`. The difference between "a component used as a type" and "a schema used as a field" is the source of several surprises, and it is worth checking both positions when a contract changes.

A message with no fields at all is legal protobuf but awkward in C#: `new Empty()` carries nothing, and a consumer cannot tell it apart from any other empty message. The `has_value` member is the mapper's way of making "this response has no declared body" explicit rather than silent. It is not a field the document declared, and it is not written on the wire unless set to `true`.

What determines the shape

The decision order in `MapProperty` is the specification of this chapter, and it is worth quoting as a list:

1. an arbitrary JSON value marker → `google.protobuf.Value`;
2. a `$ref` → message or enum, by the referenced schema;
3. `type: array` → a repeated field (chapter 10);
4. an inline `enum` → `<Property>Enum` (chapter 11);
5. an object with properties → a nested message when it is a named component; an inline object is named by the mapper but not emitted in 1.5.0 (chapter 17);
6. an object without properties → `google.protobuf.Struct`;
7. anything else → a scalar (chapter 7).

Nothing about a property's `description`, `example`, `default`, `readOnly` or `deprecated` changes this order. Those keywords are not carried into the descriptor, and a consumer that needs them has the OpenAPI document.

10. Arrays and Dictionaries

An OpenAPI array becomes a protobuf `repeated` field. The interesting part is the element: it may be a reference, an enumeration, an inline object, an arbitrary JSON value, or another array. This chapter gives the rule for each, and the two places where a repeated field cannot say what the document said.

The element decides

Array's `items`	Generated field
`$ref` to an object schema	`repeated <Message> <name>`
`$ref` to an enumeration schema	`repeated <Enum> <name>`
inline object with properties	a repeated field named after the property; the nested message is not emitted in 1.5.0 — use `$ref` (see below)
inline object without properties	`repeated google.protobuf.Struct <name>`
inline `enum`	`repeated <PropertyName>Enum <name>`
`type: string` / `integer` / …	`repeated string` / `int32` / …
the JSON-value marker	`repeated google.protobuf.Value <name>`
absent (`items` missing)	`repeated string <name>`
an array (`items.type: array`)	`repeated string <name>` (see below)

`sink.proto` shows four of these on one message:

PROTO

```
message Widget
{
    // ...
    repeated int64 counts = 12;
    repeated string matrix = 13;
    WidgetKind kind = 14;
    repeated WidgetKind tags = 15;
}
```

`counts` is a plain repeated scalar. `tags` is a repeated enumeration. `matrix` is the surprising one: its schema is an array of arrays, and a protobuf field cannot repeat a repeated field. The mapper falls through to its scalar default and produces `repeated string`, which is a real loss — the inner arrays are gone. If a contract needs a matrix, model it as a repeated message with one repeated field inside, for example `repeated Row rows`, where `Row` has `repeated double cells`.

The same fallback applies to an array with no `items` at all. It is mapped rather than refused, because an array with an unconstrained element is still an array; `string` is the mapper's documented default for an unknown type (chapter 7).

Arrays of inline objects

An array property whose items are an inline object would be named after the **property**, not after the element position:

YAML

```yaml
Widget:
  type: object
  properties:
    children:
      type: array
      items:
        type: object
        properties:
          id: { type: string }
```

The mapper derives a message named `Children` for that element. In 1.5.0 that message is not added to the emitted model, so the `repeated Children children` field points at a type the descriptor and the C# file do not contain. The generated code does not compile.

Declare the element as a named component and reference it:

YAML

```yaml
Widget:
  type: object
  properties:
    children:
      type: array
      items: { $ref: "#/components/schemas/Child" }
components:
  schemas:
    Child:
      type: object
      properties:
        id: { type: string }
```

PROTO

```proto
message Child
{
    string id = 1;
}

message Widget
{
    repeated Child children = 1;
}
```

Chapter 17 records the inline-object defect. A duplicate component schema name is still refused as `SP2005`; the missing inline message is not reported at all.

Arrays at the edges of an operation

The position changes the wrapper, not the element rule.

Position	Shape
response body is an array	`{Method}Response { repeated <Element> items = 1; }`
response array's items are inline objects	element message named `{Method}Item`
request body is an array	`{Method}Body { repeated <Element> values = 1; }`
component schema is an array	`message <Name> { repeated <Element> values = 1; }`

`ListPetsResponse` is the first row: the `200` response's schema is an array of `Pet`, so the wrapper has `repeated Pet items = 1`. The field is named `items` because the response is a payload of elements; a body message uses `values` because it is a wrapper around a single array value.

The two nullability losses

A repeated field is a sequence. It has no "unset" state distinct from empty, and it has no per-element presence. Two things therefore cannot be represented:

1. `nullable: true` on the array itself;
2. `nullable: true` on the items schema.

Both are reported as `SP2000` when `EmitDiagnosticsForUnmappableConstructs` is `true`, and both still map:

```
TEXT

warning SP2000: SharpPortico could not map 'nullable' (tags): a repeated field has no per-element
presence, so an element that was null reads back as the element's default value
```

The message names the proto field (`tags`, or `tags[]` for the element case) and says what was lost. The field remains `repeated string`; a null element becomes `""`, and a null array becomes an empty one. If that distinction matters, the contract needs a wrapper message, not a repeated field.

What is not generated

- **No `map<string, T>`.** A schema with `additionalProperties` becomes `google.protobuf.Struct`, even when the value type is declared. Chapter 17 records this.
- **No fixed-length arrays.** JSON Schema's `minItems` / `maxItems` are not carried; a 3.1 tuple's length is explicitly reported as lost (`SP1003`).
- **No multidimensional arrays.** The inner dimension collapses to `string`, as shown above.
- **No `repeated` of `optional`.** proto3 forbids it, and the mapper does not attempt it; the per-element loss is what `SP2000` describes.

The rule of thumb is the same as everywhere in this part: a repeated field carries a sequence of one type. When the document describes anything more specific — a fixed length, a position-dependent type, a nullable element — the extra meaning is reported rather than encoded.

11. Enumerations

An OpenAPI `enum` becomes a protobuf enum, and the same members become a C# `enum`. The mapping is direct enough that the interesting questions are naming, sharing and what happens when a document declares two enumerations that look alike.

The mapping

YAML

```yaml
components:
  schemas:
    WidgetKind:
      type: string
      enum: [gear, sprocket]

    Widget:
      type: object
      properties:
        kind: { $ref: "#/components/schemas/WidgetKind" }
        state:
          type: string
          enum: [on, off]
```

PROTO

```
enum WidgetKind
{
    GEAR = 0;
    SPROCKET = 1;
}

enum StateEnum
{
    ON = 0;
    OFF = 1;
}

message Widget
{
    WidgetKind kind = 14;
    StateEnum state = 26;
}
```

Three rules produced that output:

1. **A component enumeration keeps its name.** `WidgetKind` is a message-like type in the model and is referenced by `$ref`.
2. **An inline enumeration is named after its property plus `Enum`.** `state` becomes `StateEnum`.
3. **Members are PascalCase in C# and upper-case with underscores in proto.** `gear` → `Gear` / `GEAR`; `inProgress` → `InProgress` / `IN_PROGRESS`.

The C# side is a real `enum`, so a consumer writes `WidgetKind.Gear` and pays no allocation. The protobuf side is an `int` on the wire. `MessageEmitter` casts at every boundary:

C#

```
output.WriteEnum(1, (int)Kind);
Kind = (WidgetKind)input.ReadEnum();
```

Values and numbers

Declared value	C# member	Proto member	Number
`available`	`Available`	`AVAILABLE`	0
`pending`	`Pending`	`PENDING`	1
`inProgress` (third member)	`InProgress`	`IN_PROGRESS`	2
`1`	`N1`	`N1`	1
`10`	`N10`	`N10`	10

A string enumeration is numbered by declaration order starting at 0. A numeric enumeration keeps its numbers: `enum: [0, 10, 20]` becomes `N0 = 0`, `N10 = 10`, `N20 = 20`. This preserves a contract that uses numbers as stable codes, which is often the reason the enumeration is numeric in the first place.

proto3 requires the **first** member of an enum to have number 0. A numeric enumeration that starts at 1 (`[1, 2, 3]`) is emitted faithfully, as `edge.proto` shows:

PROTO

```
enum Level
{
    N1 = 1;
    N2 = 2;
    N3 = 3;
}
```

The C# types compile and round-trip, but `protoc` rejects the descriptor, because the language requires a zero value. This is one of the few places where the generated C# and the generated `.proto` disagree about validity; the CLI and the build do not add a synthetic `UNSPECIFIED = 0`, because that would change the contract's numbers. Add a zero member to the document if the descriptor has to compile.

Sharing and collisions

Two inline enumerations with the same generated name and the same members are one type and are reused. This is what a vocabulary shared by several schemas should do: `status` on `Order` and `status` on `Invoice` with identical members produces one `StatusEnum`, and both fields reference it.

Two with the same name and different members are a conflict. The mapper first derives a distinct name from the original plus the declared members (`State` + `Purged` → `StatePurged`); if that name is free, both enumerations survive and no diagnostic is raised. If the derived name is already taken by a different member set, the document is refused:

TEXT

```
error SP2007: SharpPortico cannot map 'openapi/things.yaml': the inline enumeration named 'StatusEnum' is
declared twice with different members ('available, pending' and 'sold, returned'). An inline enumeration is
named after the property declaring it, so two schemas with a property of that name collide - declare one of
them as a named schema under components
```

The remedy in the message is the right one: declare one of the two as a named component. That gives it a name you chose, keeps both member sets, and removes the dependence on a naming heuristic.

Nullability

A nullable enumeration is a singular scalar, so it takes proto3's `optional` label and gains presence members:

PROTO

```
optional WidgetKind kind = 3;
```

```C#
widget.HasKind;    // was it sent?
widget.Kind;       // the value, or the default (the first member)
widget.ClearKind();
```

A `$ref` to an enumeration behaves the same as an inline one, because the field kind is `Enum` after the reference is resolved. A `$ref` to an object does not, because a message field already has presence. Chapter 12 gives the full rule.

In the gateway

The proxy's generated JSON writer emits an enumeration as its **number**, not its name:

```C#
sb.Append((int)msg.Kind);
```

and the reader calls `GetInt32()` before casting. A legacy service that expects `"gear"` in JSON will not understand the gateway's `0`, and a response containing `"gear"` will fail to read. This is a real limitation of proxy mode for enumerated fields, not an option; chapter 23 describes it with the rest of the JSON mapping. If the legacy API's enumerations are strings, the gateway is not a drop-in bridge for those fields.

What is not generated

- **No `UNRECOGNIZED` member.** protoc's C# output adds one; SharpPortico does not, because the contract did not declare it and every switch over the enum would need a case the document never mentioned.
- **No aliases.** Two members with the same number are legal proto3 only with `allow_alias`, which is not emitted. A numeric enumeration with duplicate numbers will not compile as a descriptor.
- **No description.** OpenAPI `description` on the enum or its members is not carried into the `.proto`.
- **No switch to plain C# enums.** `MapEnumsToProtobufEnums` exists on the attribute but is not read (chapter 5); enumerations are always protobuf enums.

12. Nullability and Presence

A singular proto3 scalar has no presence of its own: an absent field and a field set to `0` are the same on the wire. SharpPortico restores the distinction the contract asked for by emitting proto3's `optional` label and generating `Has…`/`Clear…` members, the way `protoc` does.

The difference in one table

For `retries: { type: integer, nullable: true }`:

Situation	Before 1.5.0	1.5.0
JSON/C# value is `null`	`Retries = 0`, indistinguishable from `0`	`HasRetries == false`, `Retries` reads `0`
value is `0`	`Retries = 0`	`HasRetries == true`, `Retries == 0`
field omitted by the sender	`Retries = 0`	`HasRetries == false`
on the wire	a `0` is never written	a set `0` is written; an absent field is not
gateway	a `null` parameter was dropped	the parameter is sent whenever it was set

That is the whole point: "keep the current value" and "set it to zero" are different requests, and a service that receives `0` for both cannot tell them apart.

What becomes optional

OpenAPI schema	`.proto`	Generated C#
`type: integer, nullable: true`	`optional int32 retries = 3;`	`int Retries`, `bool HasRetries`, `void ClearRetries()`
`type: string, nullable: true`	`optional string nickname = 4;`	`string Nickname` + presence
`type: boolean, nullable: true`	`optional bool enabled = 5;`	`bool Enabled` + presence
`type: string, format: byte, nullable: true`	`optional bytes blob = 6;`	`ByteString Blob` + presence
inline or `$ref` enum, `nullable: true`	`optional StatusEnum status = 7;`	`StatusEnum Status` + presence
`$ref` to an object, nullable	`Pet data = 1;` unchanged	already a presence type
`type: array, nullable: true`	`repeated …` unchanged	**loss**, `SP2000`
`items` nullable	`repeated …` unchanged	**loss**, `SP2000`

Two kinds never take the label: a **message** field is absent until it is set, and protobuf has no way to write "a message set to null"; a `Timestamp` would be a message too, though this version's mapper never produces one (chapter 7). Everything else singular — enums, strings, bytes, numbers and booleans — takes it.

The generated members

```C#
private int _value_Retries;
private bool _has_Retries;

public bool HasRetries => _has_Retries;

public void ClearRetries()
{
    _has_Retries = false;
    _value_Retries = default;
}

public int Retries
{
    get { return _value_Retries; }
    set { _value_Retries = value; _has_Retries = true; }
}
```

A string or `bytes` field uses a nullable backing field and returns the protobuf default from the getter:

```C#
private string? _field_Nickname;
private bool _has_Nickname;

public string Nickname
{
    get { return _field_Nickname ?? ""; }
    set { _field_Nickname = value; _has_Nickname = true; }
}
```

The names match `protoc`'s C# output, so code written against a `protoc`-generated message reads the same here. Writing the property raises the flag; `Clear…` resets both the value and the flag.

Wire and size

`CalculateSize`, `WriteTo` and `MergeFrom` ask the flag, not the value:

```C#
// a set 0 is written; an absent field is not
if (HasRetries) { output.WriteRawTag(24); output.WriteInt32(Retries); }
```

On the reading side, a field present in the payload sets the flag — even when its value is the default. A field absent from the payload leaves it unset. This is exactly how `optional` behaves in `protoc`-generated code, and it is why the descriptor and the classes agree.

`Equals`, `GetHashCode` and `Clone` also use the flag, so `new Thing()` and `new Thing { Retries = 0 }` are not equal. That is the C# expression of the same distinction.

On the server

The flag is the question a `PATCH`-style endpoint actually asks:

```C#
public override Task<UpdateThingResponse> UpdateThingAsync(UpdateThingRequest request, ServerCallContext context)
{
    if (request.HasRetries)
    {
        // Sent, even when it is 0 — which a "!= 0" test used to swallow.
        _thing.Retries = request.Retries;
    }

    if (request.HasNickname)
    {
        _thing.Nickname = request.Nickname;   // may be an explicit ""
    }

    return Task.FromResult(new UpdateThingResponse { Data = _thing });
}
```

`request.Retries != 0` is the wrong test for an optional field: it treats an explicit zero as "not sent". Use `HasRetries` whenever the contract says the field may be null.

On the client

```C#
var thing = new Thing { Nickname = "Atlas" };

thing.HasNickname;      // true
thing.HasRetries;       // false — never set
thing.Retries;          // 0, the protobuf default, no presence behind it
thing.Retries = 0;      // now it has presence
thing.HasRetries;       // true
thing.ClearRetries();   // back to absent
```

A non-nullable property in the same message has no `Has…` member. Presence is a fact about the contract, not a default applied to every scalar.

In the gateway

The proxy uses the same flag in both directions. On the way out, a set field is sent even when its value is the default, and an unset one is omitted; on the way back, a JSON `null` leaves the field unset rather than writing `0` or `""`. The generated code reads:

```C#
if (request.HasRetries)
{
    queryParams["Retries"] = Convert.ToString(request.Retries, CultureInfo.InvariantCulture) ?? "";
}
```

and, for a response, only assigns when the JSON value is not `null`. Chapter 23 has the full JSON rules.

OpenAPI 3.1

The 3.1 spelling of a nullable string is a type array:

```YAML
callsign: { type: [string, "null"] }
```

The downgrade rewrites it to `type: string` plus `nullable: true`, so the result is identical:

```proto
optional string callsign = 2;
```

A type array with **two non-null types** is not a nullable type; it is a union, and it maps to `google.protobuf.Value` (chapter 6). The rewrite distinguishes the two by counting the non-null entries.

When presence cannot be represented

A nullable array or element. `repeated` has no unset state and no per-element presence. The field still maps, and `SP2000` states the loss:

```
warning SP2000: SharpPortico could not map 'nullable' (tags[]): a repeated field has no per-element
presence, so an element that was null reads back as the element's default value
```

A presence member the schema already declares. A nullable `more` needs `HasMore` and `ClearMore`; a property literally named `hasMore` would produce the same member twice. That is refused, because no generated code could satisfy both:

```
error SP2008: SharpPortico cannot map 'openapi/things.yaml': property 'more' of schema 'Thing' is
nullable, so its message needs presence members named 'HasMore' and 'ClearMore', but 'HasMore' is
already a member of 'Thing'. Rename one of the two
```

Both cases are about the same limit: presence is a property of a singular field, and a contract that asks for it elsewhere is asking protobuf for something the format does not have.

Why not wrappers?

The alternative to `optional` is the `google.protobuf.*Value` wrapper types (`Int32Value`, `StringValue`, ...). They also carry presence, and they were the standard answer before proto3 added `optional`. The mapper does not use them because they change the JSON shape (`{"value": 0}` instead of `0`), add an allocation per set field, and require `google/protobuf/wrappers.proto`. `optional` is the modern spelling of the same idea, and it keeps the wire format identical to a plain scalar apart from the tag being written for a default value.

13. Operations and RPC Kinds

Every operation under `paths` becomes one RPC. Most are unary; a hint or a size heuristic can make one streaming. This chapter states how an operation is chosen, named, typed and given its kind.

One operation, one RPC

`MapPathOperations` walks `document.Paths` in document order, then each path item's operations, and calls `MapOperation` for every one. No verb is skipped: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, `OPTIONS` and `TRACE` all become RPCs. The HTTP method is recorded on the RPC model (`HttpMethod`) and used by the CLI report and the gateway, not by the protobuf descriptor.

The descriptor carries only the RPC name and the message types:

```proto
service PetService
{
    rpc ListPets (ListPetsRequest) returns (ListPetsResponse);
    rpc CreatePet (CreatePetRequest) returns (CreatePetResponse);
    rpc GetPet (GetPetRequest) returns (GetPetResponse);
}
```

There is no `google.api.http` annotation and no path in the descriptor. A consumer that needs the REST mapping reads the OpenAPI document or the CLI report; the generated C# method descriptors expose the service name and method name only.

Naming

The RPC name is `SanitizePascal(operationId)` (chapter 8). Without an `operationId` the name is `Op{index}_{METHOD}`, sanitised: the first operation in `edge.yaml` has no id, and the result is `Op0POST`.

The two wrapper messages are named `{Rpc}Request` and `{Rpc}Response` in every case, including streaming. A streaming RPC still has one request type and one response type; `stream` changes how many messages travel, not which messages they are.

The four kinds

Kind	`.proto`	Trigger
unary	`rpc X (Req) returns (Resp);`	default
server-streaming	`rpc X (Req) returns (stream Resp);`	`x-grpc-streaming: server`, or a cursor/next-token parameter
client-streaming	`rpc X (stream Req) returns (Resp);`	`x-grpc-streaming: client`, or a large `POST` body
bidirectional	`rpc X (stream Req) returns (stream Resp);`	`x-grpc-streaming: bidi`

The extension is read from the operation:

```yaml
post:
  operationId: uploadReadings
  x-grpc-streaming: client
```

Values are matched case-sensitively against `client`, `server` and `bidi`; anything else, including `unary`, sets the kind to unary. `RespectStreamingHints = false` ignores the extension entirely, which is the switch to reach for when a legacy document carries the annotation for another tool.

The large-body heuristic

With `RespectStreamingHints` on and no explicit hint, a `POST` whose request body is estimated at or above `LargePayloadStreamingThresholdBytes` (default `1_000_000`) becomes **client-streaming**.

The estimate is deliberately crude, because the generator never sees a payload:

```C#
foreach (var media in body.Content.Values)
{
    total += media?.Schema?.Type == "object" && media.Schema.Properties is { } props
        ? props.Count * 64L
        : 256;
}
```

An object body is assumed to be 64 bytes per property; anything else is 256 bytes. A body with 16,000 properties crosses the default threshold. In practice the heuristic fires rarely, and when it does the document's shape — not a measured size — is the reason. If client-streaming is wrong for an operation, set `x-grpc-streaming: unary` explicitly or turn `RespectStreamingHints` off.

Only `POST` is considered. A large `PUT` or `PATCH` stays unary, because those verbs usually describe a single replacement rather than a feed.

Pagination changes the kind

When `DetectPagination` is on and the request contains a field named like the cursor or next-page-token parameter, a unary RPC becomes **server-streaming** unless the document already supplied a streaming hint:

```YAML
parameters:
  - name: cursor
    in: query
    schema: { type: string }
```

```PROTO
rpc ListEvents (ListEventsRequest) returns (stream ListEventsResponse);
```

A page/limit pair does **not** change the kind; it only marks the RPC as paginated, which the proxy and report can use. Chapter 14 covers the detection rules and why the cursor is treated differently.

Request and response shape

The request message contains, in order:

1. one field per path, query and header parameter, in the order the operation declares them;
2. the request body, as `body`, when there is one.

The response message contains the success response's payload:

Success response	Response message
`$ref` schema	`{ <Message> data = 1; }`
inline object	`{ <Method>Data data = 1; }`
array	`{ repeated <Element> items = 1; }`
`application/octet-stream`	`{ bytes body = 1; }`
no content	`{ bool has_value = 1; }`

Success response	Response message
no success response at all	`{ <Method>Error error = 1; }` when `EmitGoogleRpcStatusWrapper` is on

"Success" means the first of `200`, `201`, `204` or `default`, in that order. A `204` with no content is a legal response and produces `has_value`; it is not an error. An operation with no response section at all is the case the error wrapper exists for.

Streaming on the client

The nested `Grpc.Core` client exposes the four shapes with the usual signatures:

Kind	Client method
unary	`AsyncUnaryCall<Resp> XAsync(Req request, …)`
server	`AsyncServerStreamingCall<Resp> X(Req request, …)`
client	`AsyncClientStreamingCall<Req, Resp> X(…)`
bidi	`AsyncDuplexStreamingCall<Req, Resp> X(…)`

Only the unary method has `Async` in its name; the streaming methods are named after the RPC, which is what `Grpc.Core`'s own generated clients do. The convenience client adds `XStream(…)` methods that return the same call objects.

The server base is shaped by the kind:

Kind	Method to override
unary	`Task<Resp> XAsync(Req request, ServerCallContext context)` (plus `XHandler` and `X`)
server	`Task X(Req request, IServerStreamWriter<Resp> responseStream, ServerCallContext context)`
client	`Task<Resp> X(IAsyncStreamReader<Req> requestStream, ServerCallContext context)`
bidi	`Task X(IAsyncStreamReader<Req> requestStream, IServerStreamWriter<Resp> responseStream, ServerCallContext context)`

Server-streaming and bidi defaults complete without writing anything, rather than throwing `Unimplemented`, because a stream that ends immediately is a valid empty implementation. Client-streaming and unary throw `Unimplemented`, so a forgotten override fails loudly.

What an operation loses

- **The HTTP method and path.** They are not in the descriptor. A gRPC consumer cannot tell `GET /pets` from `POST /pets` if both were mapped with the same request and response types.
- **Status codes.** Success status is not represented; error statuses become `RpcException` at run time, and the mapping from HTTP to gRPC exists only in the gateway (chapter 22).
- **Headers as a group.** A header parameter becomes an ordinary request field. Reserved headers such as `Authorization` are not treated specially by the mapping; authentication is a separate concern (chapter 21).
- **Media types.** Only the first media type with a schema is read; `application/json` and `application/octet-stream` get special treatment, and the rest are ignored (chapter 15).
- **Operation metadata.** `summary`, `description`, `tags`, `deprecated` and `externalDocs` are not emitted.

These are not oversights to work around; they are the boundary between an HTTP contract and a protobuf one. The next chapter goes one level down, into the parameters and bodies that fill the request message.

14. Parameters, Bodies and Pagination

A request message is assembled from three sources: parameters, the request body, and the pagination heuristic. This chapter describes the order, the shapes, and the places where HTTP information has nowhere to go.

Parameters become fields

Every parameter with a schema is mapped exactly like a property: same scalar table, same array rule, same `$ref` handling, same presence rule. The parameter's location (`path`, `query`, `header`, `cookie`) does not change the type.

`edge.yaml` declares a header, a cookie and a query parameter:

YAML

```yaml
parameters:
  - name: X-Trace-Id
    in: header
    schema: { type: string }
  - name: session
    in: cookie
    schema: { type: string }
  - name: URL
    in: query
    schema: { type: string }
```

PROTO

```proto
message Op0POSTRequest
{
    string x_trace_id = 1;
    string session = 2;
    string u_r_l = 3;
}
```

The original location is not recorded in the descriptor. A consumer that needs to know whether `session` was a cookie or a header reads the OpenAPI document; the generated client simply has a field.

Two parameter details are ignored:

- **`required`**: a missing parameter is the field's default. There is no validation, and no presence unless the schema says `nullable`.
- **`style`**, **`explode`**, **`allowEmptyValue`**, **`example`**: serialization and documentation hints, not types.

A parameter array becomes a repeated field, which is the closest protobuf equivalent to a repeated query parameter. How the legacy service expects it encoded (`ids=1&ids=2` versus `ids=1,2`) is not represented; the gateway does not send a repeated parameter at all, and a top-level repeated scalar makes its generated file fail to compile (chapters 22 and 23).

The request body

The body is mapped after the parameters, so its field number is `parameterCount + 1`.

Request body content	Field
any `application/octet-stream` entry	`bytes body`
first entry whose schema is a `$ref`	`<ReferencedMessage> body`
first entry whose schema is a free-form object	`google.protobuf.Struct body`
first entry whose schema is an arbitrary JSON value	`google.protobuf.Value body`
first entry with an inline object schema	`<Method>Body body`

Request body content	Field
first entry with an inline array schema	`<Method>Body body`, and that message has `repeated <Element> values`
first entry with a scalar schema	`<Method>Body body`, and that message has `<scalar> value`

The `application/octet-stream` check runs first and is not order-dependent. Everything else takes the first media type entry that has a schema; a document listing `application/json` and `application/xml` therefore uses the JSON one if it appears first. The others are ignored, rather than represented as alternatives.

`required: true` on the body does not make the field mandatory. A message field is always optional on the wire; the generated server sees `null` when no body was sent, which is the check `PetServiceImpl` performs:

```C#
if (request.Body is null) throw new RpcException(new Status(StatusCode.InvalidArgument, "body required"));
```

No parameters and no body

An operation with nothing to send still needs a request message. The mapper inserts a single member:

```PROTO
message GetThingByIdRequest
{
    bool has_value = 1;
}
```

The field exists so that the message is not empty, and so a caller can distinguish a request that carried `has_value = true` from one that carried nothing — though in practice the distinction is unused. A response with no declared content gets the same member.

Pagination detection

`DetectPagination` (default `true`) looks at the **proto names** of the request fields after they are mapped:

```TEXT
page, limit, cursor, next_page_token
```

The names are configurable through `PaginationPageParameter`, `PaginationLimitParameter`, `PaginationCursorParameter` and `PaginationNextPageTokenParameter`. Matching is case-insensitive, which matters because proto names are lower-case: `nextPageToken` maps to `next_page_token` and matches the default.

Three outcomes follow:

Fields present	Result
`page` and/or `limit`	`RpcModel.HasPagination = true`; kind unchanged
`cursor` or `next_page_token`	`HasPagination = true`; kind becomes server-streaming unless an explicit hint exists
none	no change

The distinction is intentional. A page/limit API returns a bounded page and expects the caller to ask again; that is still one request and one response, so it stays unary. A cursor or token API is naturally a feed; server-streaming lets one call deliver successive pages. The heuristic cannot know whether the service actually supports that, so an explicit `x-grpc-streaming: unary` is the way to keep a cursor-based operation unary.

Pagination detection does **not** change any message. There is no synthesized `next_page_token` field, no page wrapper, and no automatic loop. It marks the RPC and may change its kind; the implementation still decides what a page means.

Why detection is name-based

The alternative is to infer pagination from response shape: an object with `items` and `total`, or a field called `next`. That would fire on any list response and produce streaming RPCs nobody asked for. A name is a claim the contract makes deliberately; a shape is a coincidence. The mapping follows the claim and gives you four properties to rename if your contract uses different words.

Order and numbers

Field numbers are assigned in the order the mapper adds fields:

1. parameters, in the order they appear in the operation's `parameters` array;
2. the body, last.

That order is stable for a given document, and it is part of the wire contract. Reordering parameters in the OpenAPI document renumbers the fields, which breaks compatibility with any consumer built against the previous descriptor. Treat parameter order as you would treat a `.proto` field number: append, do not rearrange.

The mapper does not deduplicate parameter names. Two parameters called `id` — one in the path, one in the query — produce two fields named `Id` with consecutive numbers, and the generated C# does not compile. Rename one in the document; the mapping cannot choose which one the caller meant.

What HTTP adds that protobuf cannot

A request message is a value. HTTP requests have properties a value does not:

HTTP feature	In the generated contract
path template	not present; the proxy keeps it on the RPC model
query string encoding	not present; repeated fields have no encoding convention
header names	only as field names in the message, without the header semantics
cookies	ordinary fields; no cookie jar
content negotiation	not present; the first media type was chosen at generation time
`Content-Length`, `Transfer-Encoding`	transport concerns, handled by gRPC
conditional requests (`If-Match`, …)	ordinary string fields if declared; no semantics

When a contract depends on one of these, the gRPC consumer has to reconstruct the convention. That is a legitimate design: the contract says what the values are, and the transport is gRPC's. It becomes a problem only when the legacy service's behaviour depends on the convention itself, which is the case proxy mode exists to handle.

15. JSON and Media Types

gRPC does not send JSON. A request body declared as JSON becomes a protobuf message, and the JSON document is only involved when a client or the gateway converts. This chapter covers which media type the mapper reads, what a JSON schema becomes, and what is lost when JSON's flexibility meets protobuf's types.

Choosing a media type

`MapRequestBodyField` and `MapSuccessResponse` each select one schema from a `content` map:

1. if `application/octet-stream` is present, it wins, and the field is `bytes` (chapter 16);
2. otherwise, the first entry — in the document's order — whose schema is not null is used;
3. entries without a schema are skipped;
4. all other entries are ignored.

There is no preference for `application/json` over `application/xml` at this step. A document that lists XML first produces the XML schema's messages if that schema is the first with a schema at all. The recommendation is to put the media type you mean first, or to declare only one.

`application/problem+json`, `application/merge-patch+json` and vendor types such as `application/vnd.api+json` are treated as ordinary media types: their schema is read, and the JSON dialect implied by the name is not interpreted.

What a JSON schema becomes

JSON and protobuf agree on more than they disagree:

JSON Schema	Protobuf
object with `properties`	message; an inline object under a property is not emitted in 1.5.0 (chapter 17)
array	repeated field
string / number / boolean	scalar
nested object reached through `$ref`	message (inline objects are not emitted; chapter 17)
`$ref`	message or enum reference
`enum`	protobuf enum
object with no properties	`google.protobuf.Struct` in a field position; a named component becomes a placeholder message with `has_value`
union of several types	`google.protobuf.Value`

The disagreements are where this manual spends its time: nullability (chapter 12), arrays of arrays (chapter 10), unions (chapter 9), and validation keywords (chapter 17).

JSON-specific constructs that are ignored

Construct	Why it does not map
`discriminator`	protobuf has no tagged union; `oneOf` is flattened with a `kind` string
`readOnly` / `writeOnly`	a protobuf message is used in both directions; omitting a field would make the same schema two types
`xml`	an XML serialization hint
`example` / `examples`	documentation

Construct	Why it does not map
`default`	protobuf has no schema-level defaults; a consumer's default is the type's zero value
`pattern` , `minLength` , `maximum` , …	validation, not type
`propertyNames` , `dependentRequired`	object-level validation
`$schema` , `$id` , `$anchor`	JSON Schema bookkeeping

A `discriminator` is the one that most often surprises a reviewer. OpenAPI's `oneOf` plus `discriminator` is a tagged union; the mapper produces a flat message with a `string kind` plus the first object choice's fields. The discriminator's property name and mapping are not emitted, so a consumer cannot use the generated `Kind` to decide which fields are meaningful. If a contract depends on a real union, the honest protobuf contract is one message per variant with a `oneof` — which this generator does not produce from OpenAPI.

JSON in the gateway

Proxy mode does serialize and parse JSON at run time, with generated code rather than reflection. Its rules are not the OpenAPI document's rules; they are the legacy service's JSON, and they are covered in chapter 23. Three facts are worth having here because they affect the contract:

- Property names are the **proto** names (`pet_id`), not the OpenAPI names (`petId`) and not the C# names (`PetId`).
- Enumerations are written and read as **numbers**.
- A `bytes` field is base64 text.

If the legacy API uses camelCase JSON and string enumerations — which most do — the gateway's output will not match it. Chapter 23 states the limits plainly.

There is a sharper limit for free-form data. The gateway's JSON helpers are generated per message in the model, and `google.protobuf.Struct` and `google.protobuf.Value` are not model messages: a proxy generated for a contract containing either references a serializer that does not exist, and the generated file does not compile. A contract with free-form objects or unions can use the messages, the server and the client, but not proxy mode in 1.5.0. Chapter 17 records this with the other losses.

Binary over gRPC

Protobuf's own encoding is binary and compact; there is no JSON step between the generated message and the wire. A `string` field is UTF-8, a `bytes` field is raw octets, and numbers are varints or fixed-width depending on their type. That is why the mapping can be reflection-free and why a payload does not grow when it crosses the gRPC boundary.

The practical consequence is that `application/json` in the document describes the **legacy** API's media type, not the generated contract's. A gRPC consumer sends a message; it does not send a JSON document. Only the gateway converts back.

16. Binary Bodies and Files

OpenAPI describes binary payloads in several ways, and protobuf has exactly one type for them: `bytes`. This chapter covers which source constructs become `bytes`, what a download looks like as a message, and why binary payloads are the weakest part of proxy mode.

What becomes `bytes`

OpenAPI	Proto
`type: string, format: binary`	`bytes`
`type: string, format: byte`	`bytes` (base64 in JSON already)
`type: string, contentEncoding: base64` (3.1)	`bytes`
a request body of `application/octet-stream`	`bytes body = 1`
a response of `application/octet-stream`	`bytes body = 1`
`type: string` with no format	`string` — not bytes

The `format: binary` rule matters most. In 3.0, a file upload is usually described as `type: string, format: binary`, which does **not** mean a string with strange characters; it means an octet stream. Mapping it to `string` would force a UTF-8 assumption the contract never made. `sink.yaml` declares a binary response and gets:

```proto
message UploadPhotoResponse
{
    bytes body = 1;
}
```

A 3.1 document may use `contentEncoding: base64` on a string. The downgrade step rewrites it to `format: byte`, which the mapper maps to `bytes`, so the two versions converge (chapter 6).

A binary response is a message

A successful `application/octet-stream` response does not become a bare `bytes` RPC return; it becomes a message with one `bytes` field. That keeps the convention that every RPC returns a message, which is what the generated client and server code assumes, and it leaves room for a future metadata field without changing the RPC signature.

The consequence is that the consumer unwraps one level:

```C#
var response = await client.UploadPhotoAsync(new UploadPhotoRequest { Id = id, Body = photo });
byte[] stored = response.Body.ToByteArray();
```

`Body` is Google.Protobuf's `ByteString`. `ToByteArray()` copies; `Span` / `Memory` access is available on modern runtimes without the copy. The message does not carry the HTTP `Content-Type`, `Content-Disposition`, `ETag` or the filename. A contract that needs them has to declare them as response headers and map them separately — and response headers other than the success payload are not mapped at all, so in practice the filename is lost.

Uploads

A request body of `application/octet-stream` becomes `bytes body`. If the body is declared as a schema instead — an object with a `content` property of `format: binary` — the message has a `bytes Content` field inside a `<Method>Body` message. Both are single messages; neither is a stream.

A large binary body does **not** trigger the client-streaming heuristic. The estimate in chapter 13 counts 256 bytes for any media type without an object schema, which is below the default threshold regardless of the real size, so an upload stays unary. Streaming an upload requires an explicit `x-grpc-streaming: client`, and then each request message carries one `bytes` chunk that the service reassembles.

`multipart/form-data` is not special-cased. If the document gives it an object schema, the mapper builds that message and the file part becomes whatever its schema says; if the document only names the media type without a schema, no body field is emitted. There is no boundary parsing, no part names, and no filename.

Proxy mode and binary payloads

The gateway serializes message bodies as JSON and sends them with `application/json`. Two consequences follow from the generated code:

- an `application/octet-stream` body is a `bytes` field, not a message field, and the proxy only serializes a `body` field whose kind is `Message`. A binary request body is therefore **not sent**;
- a `bytes` field anywhere in a request message — a parameter or a body — makes the generated proxy reference a comparison that does not type-check, because the generated "is this set" test treats every non-string, non-bool field as numeric.

So a contract with binary request payloads can produce messages, a server and a client, but the generated proxy for it will not compile or will drop the payload. Chapter 23 states the same limit from the runtime side. This is a 1.5.0 boundary, not a design recommendation: a binary API needs a hand-written adapter or a gRPC-native backend.

Binary **responses** are less affected in the descriptor but equally affected in the gateway: the legacy service's octet stream arrives as an HTTP body, and the generated proxy has no `bytes` response path. The same conclusion holds.

When to use base64 instead

If the binary payload is small and the contract is JSON-shaped — an avatar thumbnail, a signature, a certificate — declaring it as `type: string, format: byte` and sending base64 inside a JSON body keeps the whole contract inside the parts of the pipeline that work. The cost is a third more bytes and an encode/decode step; the benefit is that the proxy, the presence rules and the JSON helpers all behave as documented.

For large files, neither base64 nor the generated proxy is the right tool. Serve the bytes over HTTP and keep the gRPC contract for metadata, or implement the file RPC by hand.

17. What Does Not Fit

A mapping manual that only lists what maps is a sales brochure. This chapter is the ledger of everything the generator reads and discards, everything it cannot express, and the handful of cases where the generated code does not compile at all.

Document-level constructs

OpenAPI	Status	Consequence
`info.title`, `info.version`	ignored	the protobuf `package` comes from `ProtoPackage`, not from the document
`servers`	ignored	endpoints are configured by the channel or by `HttpRestClient`; the generator emits no address
`tags`	ignored	no grouping metadata; a service is one service
`security`, `components.securitySchemes`	partly	schemes become metadata helpers and an interceptor (chapter 21); requirements and scopes are ignored
`components.responses`	partly	visible through `$ref` when a success response points at one; headers are ignored
`components.parameters`	yes	resolved through `$ref` like any other object
`components.headers`	ignored	response headers are not mapped
`components.links`	ignored	no relationship between responses
`components.callbacks`	ignored	callbacks are not operations with paths in the document model
`externalDocs`	ignored	documentation only
`webhooks` (3.1)	mapped with a loss	merged into `paths` as a `POST` operation (chapter 6)

Operation-level constructs

Construct	Status
`operationId`	RPC name (chapter 8)
`summary`, `description`	ignored
`tags`, `deprecated`, `externalDocs`	ignored
`parameters`	request fields (chapter 14)
`requestBody`	one body field (chapter 14)
`responses`	one success payload; others ignored
`security`	ignored per operation; every declared scheme is attached if auth code is enabled
`callbacks`	ignored
`servers` (operation-level)	ignored
`x-grpc-streaming`	streaming kind (chapter 13)
other `x-` extensions	ignored

The `deprecated` flag deserves a note. gRPC has an `option deprecated = true` on services, methods and fields, and the generator does not emit it. A deprecated operation becomes an ordinary RPC, indistinguishable from a supported one. If that matters, the contract should not carry the operation at all, or the descriptor should be annotated by hand.

Schema-level constructs

Construct	Status	Notes
`pattern`, `minLength`, `maxLength`, `minimum`, `maximum`	ignored	no validation is generated
`multipleOf`, `minItems`, `maxItems`, `uniqueItems`	ignored	same
`minProperties`, `maxProperties`	ignored	same
`readOnly`, `writeOnly`	ignored	one message serves both directions
`default`	ignored	protobuf's zero value is the only default
`example`, `examples`	ignored	documentation
`title`, `description`	ignored	no comments in the `.proto`
`deprecated`	ignored	no field option
`discriminator`	ignored	`oneOf` is flattened (chapter 9)
`xml`, `externalDocs`	ignored	media-specific
`nullable` (3.0)	mapped	becomes `optional` (chapter 12)
`format`	partly	only the formats in chapter 7 change the type; unknown formats are documentation
`additionalProperties: true` on `type: object`	mapped	`google.protobuf.Struct` in a field position (chapter 15)
`additionalProperties: <schema>`	ignored	the value type is not read; an object with no `properties` becomes `Struct` (chapter 10)
`additionalProperties: false`	ignored	the message is still open: unknown fields are skipped on read, and the flag is not enforced
an inline object under a property (including a property's array `items`)	**not emitted**	the mapper derives a nested message name, but 1.5.0 does not add that message to the model, so the descriptor and C# reference an undefined type; use a named component and `$ref`

The `additionalProperties: false` row is worth stating plainly: the generator does not produce a closed message. `additionalProperties: false` is a promise about the document, not about the wire, and the mapping does not pretend otherwise. Nor does the generated reader preserve unknown fields — it skips them — so a consumer that round-trips a message built by a newer contract loses the fields it did not know. This differs from `protoc`'s generated code, which keeps them in an `UnknownFieldSet`.

The inline-object row is a different kind of entry: it is an emitter defect, not a deliberate loss. A named component under `components/schemas` is emitted and can be referenced; an inline object under a property (or as that property's array `items`) is named by the mapper but never reaches `GrpcModel.Messages`. The generated field still points at that name, so the C# and the descriptor fail to resolve it. The workaround is to declare the object once as a component and use `$ref` at every position where it appears.

The losses that change behaviour

Most of the table above is metadata. Four entries change what a caller can do:

1. **One success response only.** If a `200` returns a `Pet` and a `202` returns a `Job`, the response message carries `Pet data` and the job is unreachable. The `202` is not an error; it is simply not mapped.
2. **Error bodies are unreachable.** A `400` with `application/problem+json` does not become a message. Failures arrive as `RpcException` with a status code, without the problem document.
3. **No unions.** `oneOf` and `anyOf` collapse to one message with a `kind` string (chapter 9). A caller cannot pattern-match on which variant it received.

4. **Free-form fields break the proxy.** A `Struct` or `Value` field is fine in the descriptor and in the generated messages, but the generated gateway has no serializer for it (chapter 15). The proxy file does not compile.

The first three are contract design decisions a reviewer can see and accept. The fourth is a defect in the current release, and it should be treated as one.

What the 3.1 rewrite reports

The downgrade step is the one place where the generator can see that it changed the contract, so it is the one place that warns. `SP1002` is informational: it names the constructs the rewrite had to touch. Each construct that was mapped with a loss — a multi-type array collapsed to `google.protobuf.Value`, a tuple collapsed to a `repeated` field, a webhook moved into `paths`, a non-base64 `contentEncoding` removed, a `jsonSchemaDialect` dropped — is a `SP1003` warning that says what was lost and why.

Those warnings are the closest thing to a loss report for 3.1 input. For 3.0 input there is no equivalent, because nothing is rewritten: `discriminator`, `additionalProperties: false` and ignored headers are dropped silently. The tables above are the only complete record.

A worked example: `sink.yaml`

`sink.yaml` is the fixture that exercises mapping breadth, not a realistic API. It declares, among other things, free-form objects, a union, binary payloads and recursion. Generated from it:

- `labels` (`additionalProperties: { type: string }`) and `meta` (`additionalProperties: true`) both become `google.protobuf.Struct`; the declared value type of `labels` is not represented;
- `anything: {}` becomes a `string`, because an empty schema has no type and `string` is the fallback;
- `both: oneOf: [string, integer]` also becomes a `string`; a property-level union is not the JSON-value marker, which only the 3.1 rewrite adds;
- `blob` (`format: byte`) and `photo` (`format: binary`) become `bytes`, and the octet-stream response becomes `bytes body`;
- `child` and `children` refer back to `Widget`, which is legal in both models and produces a recursive message.

The `.proto` compiles and the messages are usable. The proxy for the same document does not compile, because of the `Struct` fields. That is the shape of the current boundary: the contract side is broadly faithful, the gateway side is narrow.

How to decide whether a loss matters

Three questions, in order:

1. **Does a caller need the discarded information to use the service correctly?** Status codes, error bodies and variant discriminators usually do. `summary` and `tags` do not.
2. **Can the information be recovered from the request or response values?** A `next` link inside the body can stand in for response headers; a `kind` field can stand in for a discriminator if the producer fills it in.
3. **Is the workaround inside the contract?** If yes, change the document — add the fields, split the operation, declare explicit error messages. If no, the gRPC contract cannot express it, and the honest choice is to keep that operation on REST or to write the gRPC service by hand.

The generator's job is to make the first question answerable. It does that by being predictable: the same document always produces the same descriptor, and every departure from the document is one of the rules in this part.

18. Reading the Generated Message Code

The `.proto` text is a description; the C# file is the implementation. This chapter reads one generated message line by line, so that the rest of the generated surface — servers, clients, the gateway — stops looking like magic.

One class, no base class

Every message in the model becomes a `sealed partial class` implementing `Google.Protobuf.IMessage<T>`:

```C#
[global::System.CodeDom.Compiler.GeneratedCode("SharpPortico", "1.5.0")]
public sealed partial class Pet : global::Google.Protobuf.IMessage<Pet>
{
    private static readonly global::Google.Protobuf.MessageParser<Pet> _parser = new(() => new Pet());
    public static global::Google.Protobuf.MessageParser<Pet> Parser => _parser;
    public global::Google.Protobuf.Reflection.MessageDescriptor Descriptor => null!;

    private const uint Tag_1_Id = 8u;
    private const uint Tag_2_Name = 18u;
    private const uint Tag_3_Status = 24u;

    public long Id { get; set; }

    private string? _field_Name;
    public string Name
    {
        get { return _field_Name ?? string.Empty; }
        set { _field_Name = value; }
    }

    public StatusEnum Status { get; set; }
}
```

Three things are worth noticing before the details:

- `partial` is real: you may add members in another file in the same namespace, and the generator will never collide with them because it only emits the names it computed.
- There is no `MessageBase` and no descriptor plumbing: `IMessage<T>` is implemented directly. The wire methods are `MergeFrom`, `WriteTo` and `CalculateSize` — with the parser's overloads — plus the inherited `Descriptor`, and the class implements them itself.
- `Descriptor` returns `null!`. The generated code does not build a `FileDescriptor` or a `MessageDescriptor`; it does not need one to serialize. The last section of this chapter is about what that costs.

Field shapes

Model field	C# property
scalar, no presence	`public long Id { get; set; }`
`string`, no presence	backing field + property whose getter returns `string.Empty`
`bytes`, no presence	`public ByteString Body { get; set; } = null!;`
message, no presence	`public Pet Owner { get; set; } = null!;`
enum	`public StatusEnum Status { get; set; }`
repeated	`public RepeatedField<T> Items { get; } = new();`
proto3 `optional`	backing value, `HasX`, `ClearX`, and a property that raises the flag

Reference-typed fields default to `null`, not to an empty message. A message property that was never set is `null`, and a `bytes` property that was never set is `null`; the generated writer checks for `null` before writing either. That is why a nullable message field needs no `optional` label: absence already has a representation.

Strings are the exception: `PET` with no name and `Pet { Name = "" }` are the same protobuf value, so the property returns `string.Empty` rather than `null`, and a consumer never has to null-check a string.

Presence, in code

A field the contract marked nullable gets the full proto3 `optional` treatment:

```C#
private long _value_Age;
private bool _has_Age;

public bool HasAge => _has_Age;

public void ClearAge()
{
    _has_Age = false;
    _value_Age = default;
}

public long Age
{
    get { return _value_Age; }
    set { _value_Age = value; _has_Age = true; }
}
```

Writing `Age = 0` sets `HasAge`; never touching it leaves `HasAge` false. The writer emits the tag and a zero when `HasAge` is true, which is the whole point of the label. `ClearAge` is the only way back to absent.

For a nullable `string` or `bytes`, the backing field is nullable and the getter falls back to the protobuf default (`string.Empty` or `ByteString.Empty`), exactly as the non-optional shape does. `HasAge` still tells the two apart.

The repeated field

`RepeatedField<T>` is Google.Protobuf's own collection. It is mutable, it is never `null`, and assigning to it is not possible — callers add, clear and index it. The generated class also carries a `FieldCodec<T>` per repeated field, used by `MergeFrom` to decode packed or unpacked input:

```C#
private static readonly global::Google.Protobuf.FieldCodec<Pet> _repeated_Pets_codec =
    global::Google.Protobuf.FieldCodec.ForMessage(Tag_4_Pets, Pet.Parser);
```

For enums the codec is built with explicit casts, because the generated enum is a plain C# enum rather than protobuf's `INeedEnum`-style wrapper:

```C#
global::Google.Protobuf.FieldCodec.ForEnum(Tag_3_Statuses, x => (int)x, x => (StatusEnum)x)
```

A plain enum is easier to use from application code and works in `switch` expressions. The cost is that the enum has no descriptor and no `OriginalName`, which matters only to reflection-based tooling.

The wire methods

`WriteTo`, `CalculateSize` and `MergeFrom` are the three methods gRPC's marshaller calls, and they are the reason the class does not need reflection.

`WriteTo` writes the tag constant and then the value, skipping anything that is at its default:

```C#
if (Id != 0) { output.WriteRawTag(8); output.WriteInt64(Id); }
if (Name.Length != 0) { output.WriteRawTag(18); output.WriteString(Name); }
if (Status != 0) { output.WriteRawTag(24); output.WriteInt32((int)Status); }
```

`CalculateSize` mirrors it with `CodedOutputStream.Compute…Size`, and the two must agree exactly or the length-delimited envelope will be wrong. They are generated from the same field list, so they do.

`MergeFrom(CodedInputStream)` is a `while (ReadTag() != 0)` loop with a `switch` per tag number. A tag it does not recognise is passed to `input.SkipLastField()`, which is the point noted in chapter 17: unknown fields are not kept.

The parser is a static `MessageParser<T>` constructed once per type. The service class builds its marshallers with `Grpc.Core.Marshallers.Create<T>` when its `static readonly` method descriptors are initialized, so no marshaller is constructed per call and no reflection is involved.

Equality, cloning and text

- `Equals` compares every field; for a presence field it compares the flag **and** the value, so `new Pet { Age = 0 }` is not equal to `new Pet()`. `GetHashCode` follows the same rule.
- `Clone` copies scalars, calls `Clone()` on a message property, and **adds the same item references** for a repeated message field. Cloning a message with repeated messages is therefore shallow at the element level.
- `ToString` is a hand-written `[Pet] { Id=…, Name=…, }` string, not protobuf's text format. It is for logs.
- `IsInitialized` is always `true`: proto3 has no required fields, so there is nothing to be uninitialized.

Messages are mutable. There is no `Freeze`, no builder, and no copy-on-write. A message shared between threads and written concurrently is a data race like any other mutable object. The usual discipline applies: build a request, send it, do not mutate it afterwards.

The descriptor is null

`Descriptor => null!` is the most consequential line in the file. It means:

- **gRPC works.** `Grpc.Core` and `grpc-dotnet` marshal through `MessageParser<T>` and the three wire methods; neither asks for a descriptor.
- **`JsonFormatter` and `JsonParser` do not.** They walk `Descriptor`, and will throw.
- **Server reflection does not.** The gRPC reflection service enumerates descriptors; a service built from these messages cannot be discovered by `grpcurl` unless the `.proto` is supplied on the command line.
- **Rich error details do not compose.** `Google.Rpc.Status` and `Any` need type URLs and descriptors. The generated error wrapper is a plain `code` / `message` message for that reason (chapter 13).

If a tool needs descriptors, compile the emitted `{Service}Proto.Text` with `protoc` and use the generated types there. The descriptor string exists precisely so that path is available; it is not used by the runtime.

Extending a message

Because the class is `partial`, application code can add members in the same namespace:

```C#
namespace MyApp.Generated;

public sealed partial class Pet
{
    public string DisplayName => $"{Name} ({Id})";
}
```

This is the supported way to add behaviour. Do not add fields that look like contract fields — the generator will not know about them, they will not be serialized, and a future document change may collide with the name.

19. The Server Surface

For each service the generator emits a contract class, a base class to implement, and — when asked — DI registration and authentication helpers. This chapter covers how the pieces bind to a real server and what an implementation is expected to override.

The contract class

`public static partial class PetstoreService` holds everything that describes the service:

```C#
public const string ServiceFullName = "petstoreservice.PetstoreService";

public static readonly Grpc.Core.Method<ListPetsRequest, ListPetsResponse> Method_ListPets =
    new(Grpc.Core.MethodType.Unary, ServiceFullName, "ListPets",
        MarshallerFor<ListPetsRequest>(), MarshallerFor<ListPetsResponse>());

private static Grpc.Core.Marshaller<T> MarshallerFor<T>() where T : class, IMessage<T>, new()
    => Grpc.Core.Marshallers.Create<T>(serialize, deserialize);

public static ServerServiceDefinition BindService(PetstoreServiceBase serviceBase) { … }

public static void BindService(ServiceBinderBase serviceBinder, PetstoreServiceBase serviceImpl) { … }
```

The method name in the descriptor is the RPC name, not the C# method name: `ListPets`. The service name is `{ProtoPackage}.{ServiceName}`. Both matter to a gRPC caller. The method name comes from the document; the package and service name come from options (chapter 5).

`MarshallerFor` is built on `Grpc.Core.Marshallers.Create`, which takes a serialize and a deserialize delegate. The generated delegates write to a `MemoryStream` through `CodedOutputStream` and read through `CodedInputStream`; no reflection and no descriptor is involved (chapter 18). Each method descriptor is `static readonly`, so the marshaller pair is created once per RPC at type initialization.

There are two `BindService` overloads because there are two server hosts:

Overload	Used by
`BindService(PetstoreServiceBase)` → `ServerServiceDefinition`	`Grpc.Core.Server`
`BindService(ServiceBinderBase, PetstoreServiceBase)` → `void`	`grpc-dotnet`'s `MapGrpcService<PetstoreServiceBase>()`

The base class carries `[BindServiceMethod(typeof(PetstoreService), "BindService")]`, which is how grpc-dotnet finds the second overload. The first is what a `Grpc.Core` host calls:

```C#
var server = new Server
{
    Services = { PetstoreService.BindService(new PetstoreServiceImpl()) },
    Ports = { new ServerPort("localhost", 50051, ServerCredentials.Insecure) }
};
server.Start();
```

The base class

`PetstoreServiceBase` is abstract and has one virtual member per RPC, in the shape its kind requires:

Kind	Member to override	Default
unary	`Task<Resp> XAsync(Req request, ServerCallContext context)`	throws `Unimplemented`
server	`Task X(Req request, IServerStreamWriter<Resp> stream, ServerCallContext context)`	completes
client	`Task<Resp> X(IAsyncStreamReader<Req> stream, ServerCallContext context)`	throws `Unimplemented`
bidi	`Task X(IAsyncStreamReader<Req> in, IServerStreamWriter<Resp> out, ServerCallContext context)`	completes

The unary shape also carries two forwarding members: `XHandler` (used by the `Grpc.Core` binding) and `X` itself (used when grpc-dotnet binds by the RPC's own name). Both forward to `XAsync`, so overriding `XAsync` is enough; overriding `X` would be bypassed by the `Grpc.Core` path.

The defaults are deliberate. A unary RPC that was never implemented should fail loudly, because a caller expects a response. A streaming RPC that was never implemented should end cleanly, because an empty stream is a valid answer and because a server-streaming handler that throws `Unimplemented` after the first message is harder to diagnose than a stream that simply ends.

The `PetServiceImpl` in the sample overrides three unary methods and nothing else:

```C#
public override Task<GetPetResponse> GetPetAsync(GetPetRequest request, ServerCallContext context)
{
    if (!_pets.TryGetValue(request.PetId, out var pet))
        throw new RpcException(new Status(StatusCode.NotFound, $"pet {request.PetId} not found"));
    return Task.FromResult(new GetPetResponse { Data = pet });
}
```

Errors and the call context

A handler reports failure by throwing `RpcException` with a `Status`. That is the only error channel; there is no error return type and no error field on a response message. `StatusCode.InvalidArgument`, `NotFound`, `PermissionDenied`, `FailedPrecondition` and `Internal` are the usual choices; the HTTP status the OpenAPI document declared does not choose one for you (chapter 22 maps in the other direction, gateway only).

`ServerCallContext` provides:

- `CancellationToken` — cancelled when the client cancels or the deadline expires; pass it to your I/O;
- `RequestHeaders` and `ResponseTrailers` — the gRPC metadata, including the auth headers in chapter 21;
- `Peer`, `Host`, `Method` and `Deadline` — transport information;
- `WriteResponseHeadersAsync` — for server-streaming handlers that need to send metadata before the first message.

For server-streaming, write to `IServerStreamWriter<Resp>` and return when the stream is done. For client-streaming, `IAsyncStreamReader<Req>.MoveNext()` until it returns false, then return one response. None of these helpers buffer the stream; backpressure is the transport's.

Hosting on ASP.NET Core

The `ServiceBinderBase` overload is what `MapGrpcService` uses:

```C#
builder.Services.AddGrpc();
builder.Services.AddScoped<PetstoreServiceBase, PetstoreServiceImpl>();

var app = builder.Build();
app.MapGrpcService<PetstoreServiceBase>();
```

`AddGrpc` supplies the HTTP/2 endpoint and the marshallers; the generated binder supplies the methods. Any dependency injection the implementation needs arrives through its constructor, because grpc-dotnet resolves the service type per call from the request scope.

The generated `ServiceCollectionExtensions.AddSharpPorticoPetstoreService()` is for the **client** side only. It registers a singleton `PetstoreServiceClient` built from a `GrpcChannel` that the application has already registered:

```C#
builder.Services.AddSingleton(GrpcChannel.ForAddress("https://localhost:5001"));
builder.Services.AddSharpPorticoPetstoreService();
```

There is no generated server registration, because the server type is your implementation, not a generated one. A `AddSharpPorticoPetstoreService` that guessed a lifetime for `PetstoreServiceImpl` would be wrong for half its users.

Threading and lifetime

A handler may be called concurrently. The base class holds no state, and neither do the generated messages, so concurrency safety is entirely your implementation's concern. Both the `Grpc.Core` and grpc-dotnet hosts invoke handlers on the thread pool and do not serialize calls to one service instance.

The sample sidesteps the question by using a `Dictionary<long, Pet>` guarded by nothing — fine for one client, wrong for a server. A real implementation needs a concurrent collection or a lock, and a scoped service registered as scoped should not be stored in a singleton.

Server-streaming and bidi handlers stay alive for the duration of the stream. They must observe `context.CancellationToken`, or a client that disconnects leaves a loop writing to a dead stream until the next write fails.

What the server surface does not include

- **No HTTP transcoding.** `MapGrpcService` serves gRPC (and gRPC-Web if configured); it does not serve REST. The OpenAPI paths are not routes on this server. For that, see the proxy in part V.
- **No authorization policy.** The auth helpers generate metadata and an interceptor for the client; the server sees headers and decides. Chapter 21 covers what is and is not generated.
- **No health, reflection or metrics service.** Add `Grpc.HealthCheck`, `Grpc.AspNetCore.Server.Reflection` (which cannot work without descriptors, chapter 18) or OpenTelemetry yourself.
- **No interceptors.** A server interceptor is registered with the host, not generated; the generated binder does not stand in the way of one.

The split is intentional: the generator owns the contract, and the host owns the process. The next chapter is the other side of the same split, on the client.

20. The Client Surface

There are two clients per service: a low-level one that mirrors what `protoc` generates, and a convenience one that removes the request wrapper when the contract allows it. Both are in the same file and both use the same method descriptors.

The low-level client

The nested `PetService.PetServiceClient` derives from `Grpc.Core.ClientBase<T>` and has one method per RPC in the shape its kind requires:

```C#
public virtual AsyncUnaryCall<ListPetsResponse> ListPetsAsync(
    ListPetsRequest request, Metadata? headers = null, DateTime? deadline = null,
    CancellationToken cancellationToken = default)
    => ListPetsAsync(request, new CallOptions(headers, deadline, cancellationToken));

public virtual AsyncUnaryCall<ListPetsResponse> ListPetsAsync(
    ListPetsRequest request, CallOptions options)
    => CallInvoker.AsyncUnaryCall(Method_ListPets, null, options, request);
```

Every kind follows this pair: an overload that builds `CallOptions` from named arguments, and one that takes `CallOptions` whole. Streaming methods have the same pair with the matching call type and, for client and bidi streaming, no request argument:

Kind	Call type	Method shape
unary	`AsyncUnaryCall<Resp>`	`XAsync(Req, …)`
server	`AsyncServerStreamingCall<Resp>`	`X(Req, …)`
client	`AsyncClientStreamingCall<Req, Resp>`	`X(…)`
bidi	`AsyncDuplexStreamingCall<Req, Resp>`	`X(…)`

The `CallOptions` overload is the one to use when a deadline, a custom `WriteOptions` or per-call credentials have to be precise. The named-argument overload covers the common case.

The constructors are `internal` or `protected`:

```C#
internal PetServiceClient(CallInvoker callInvoker) : base(callInvoker) { }
internal PetServiceClient(GrpcChannel channel) : base(channel.CreateCallInvoker()) { }
```

so application code reaches the client through the convenience type's `Create` methods, and a test can reach it through a subclass that exposes the protected constructor with its own `CallInvoker`. That is why the `protected PetServiceClient(ClientBaseConfiguration)` constructor exists: `ClientBase<T>` needs it to build the per-call instance.

The convenience client

`public sealed partial class PetServiceClient : PetService.PetServiceClient` is the type most call sites use:

```C#
using var channel = GrpcChannel.ForAddress("http://localhost:50051");
var client = PetServiceClient.Create(channel);

var pet = await client.GetPetAsync(1);                // long → GetPetResponse
var name = await client.GetPetDataAsync(1);           // long → Pet
```

`Create` has two overloads: one takes a `GrpcChannel`, the other a `ChannelBase`. The second throws `NotSupportedException` unless the argument is a `GrpcChannel`, because the generated constructors need one. That is a deliberate restriction: a `CallInvoker`-based test channel does not go through `Create`.

For each unary RPC the convenience client emits up to four overloads, depending on the request and response shapes:

Condition	Overload
always	`Task<Resp> XAsync(Req request, Metadata? headers = null, CancellationToken ct = default)`
request has exactly one field	`Task<Resp> XAsync(T value, CancellationToken ct = default)`
response has exactly one field	`Task<T> X{Field}Async(Req request, CancellationToken ct = default)`
both	`Task<T> X{Field}Async(T value, CancellationToken ct = default)`

"Exactly one field" excludes the synthetic `has_value` member, so an empty request does not produce a zero-argument overload. The scalar overload's parameter name is the field name with a lower-case first letter (`PetId` → `petId`). The response overload's name is the RPC name plus the field name (`GetPetDataAsync`), not a generic `GetAsync`. A repeated response field still counts as one field: the overload returns the whole `RepeatedField<T>`, not one element.

The `headers` parameter is per-call metadata. It is also the only way to bypass the proxy's response cache on a single call (chapter 22), which is why it appears on the request-object overload rather than being folded into `CallOptions`.

For streaming RPCs the convenience client adds one method per RPC, named `XStream`:

```C#
var call = client.ListEventsStream(new ListEventsRequest());
await foreach (var page in call.ResponseStream.ReadAllAsync())
{
    // one response message per iteration
}
```

The `XStream` method returns the same call object the low-level method returns; it exists so a call site can use one client type for every kind without remembering which method carries the `Async` suffix.

The channel is yours

Nothing in the generated client chooses an address. The `servers` section of the OpenAPI document is ignored (chapter 17), and there is no generated default endpoint. The channel is where transport decisions live:

```C#
using var channel = GrpcChannel.ForAddress("https://api.example.com", new GrpcChannelOptions
{
    MaxReceiveMessageSize = 16 * 1024 * 1024,
    MaxSendMessageSize = 16 * 1024 * 1024,
    HttpHandler = new SocketsHttpHandler { EnableMultipleHttp2Connections = true }
});
```

A `GrpcChannel` is thread-safe, long-lived and expensive to build; create one per process or per target and share it. `PetServiceClient` itself is stateless apart from its `CallInvoker`, so a single instance per channel is enough. The DI extension registers exactly that: one singleton client over the resolved channel.

The generated convenience client does **not** set a deadline, and its overloads do not take one. A call with no deadline waits until the transport gives up. For anything user-facing, use the low-level client, which does expose `deadline`:

```C#
var lowLevel = new PetService.PetServiceClient(channel.CreateCallInvoker());
await lowLevel.GetPetAsync(new GetPetRequest { PetId = 1 },
    deadline: DateTime.UtcNow.AddSeconds(5));
```

The convenience overloads cannot forward a deadline; use the low-level client when it has to be per-call, or configure a default with an interceptor.

Failures

Every call can throw `RpcException`. The useful members are `StatusCode`, `Detail` (the message the server sent) and `Trailers`. A client-side mapping from status code to application error belongs in one place in your code, not at every call site:

```C#
try
{
    return await client.GetPetAsync(id);
}
catch (RpcException ex) when (ex.StatusCode == StatusCode.NotFound)
{
    return null;
}
```

`CancellationToken` cancellation surfaces as `RpcException` with `StatusCode.Cancelled`, not as `OperationCanceledException`. That is a gRPC convention, not a generator choice, and it surprises callers who are used to `HttpClient`.

The client cannot recover the HTTP status code or the error body the legacy service returned. The gateway maps some of that into gRPC status codes (chapter 22), but the mapping is one-way and lossy.

Interceptors

A client interceptor is a `Grpc.Core.Interceptors.Interceptor`, and it attaches at a `CallInvoker`:

```C#
var invoker = channel.Intercept(new LoggingInterceptor());
var client = new PetService.PetServiceClient(invoker);   // low-level client
```

The nested client's `CallInvoker` constructor is `internal`, and the generated file is compiled into your assembly, so this is legal from your own code. It is also the reason the constructor is not `public`: the intended entry point is `Create`.

The convenience client cannot wrap an intercepted invoker, because `Create` accepts only a `GrpcChannel` and a channel has no interceptor list. If a project needs an interceptor on every call, use the nested client (or write a small subclass-free factory of your own). The generated auth interceptor is written for exactly this slot and handles all five call shapes (chapter 21).

What the client does not do

- **No retries or hedging.** gRPC's retry policy is channel configuration; the generated code does not add one.
- **No serialization of query strings.** The client speaks gRPC, not REST. Even a contract generated from a document full of `GET`s sends one unary call with a request message.
- **No caching.** Only the proxy has a response cache, and only for `GET`-shaped calls.
- **No discovery.** No reflection, no service registry. The address is the channel's.

The distinction is worth keeping in mind for the next part: a generated client talks to a generated server, or to any other gRPC implementation of the same contract. It does not talk to the REST service directly. The proxy exists to bridge that last gap.

21. Authentication Helpers

The generator reads `components.securitySchemes` and emits two things for them: metadata constructors a call site can use directly, and a client interceptor that attaches the credential to every call without the call site doing anything. It does not acquire, refresh or validate a credential.

What is read

`MapAuthSchemes` runs when either `GenerateAuthMetadataHelpers` or `GenerateAuthInterceptors` is on (both default to `true`) and every scheme in `components.securitySchemes` becomes an `AuthSchemeModel`:

OpenAPI scheme	Kind	Metadata
`type: http, scheme: bearer`	`Bearer`	`authorization: Bearer <token>`
`type: apiKey, in: header`	`ApiKey` (header)	`<header name>: <key>`
`type: apiKey, in: query`	`ApiKey` (query)	`<parameter name>: <key>` in the interceptor; `x-api-key` in the helper
`type: oauth2`	`OAuth2`	`authorization: Bearer <token>`
`type: openIdConnect`	`OAuth2`	`authorization: Bearer <token>`
`type: http, scheme: basic` or any other	`Unknown`	a scheme-named metadata entry

Two things are **not** read: the operation-level `security` requirement, and the scopes in it. The generator cannot tell which RPC needs which scheme, so it assumes the service uses all of them. A document with an unauthenticated health operation and an authenticated list operation gets the same treatment for both.

The metadata helpers

For each scheme the contract class's sibling `PetServiceAuth` gets one method:

```C#
public static class PetServiceAuth
{
    public static Metadata CreateBearerTokenMetadata(string token)
        => new() { { "authorization", $"Bearer {token}" } };

    public static Metadata CreateApiKeyMetadata(string key)
        => new() { { "x-api-key", key } };
}
```

The metadata key is lower-cased, because gRPC requires metadata keys to be lowercase. The helper's name is fixed per kind, except for `Unknown`, which uses the scheme name (`CreateOAuthMetadata` for a scheme named `oauth`).

Use a helper when one call needs a credential and the rest do not:

```C#
var headers = PetServiceAuth.CreateBearerTokenMetadata(token);
await client.ListPetsAsync(new ListPetsRequest(), headers);
```

The helper allocates a fresh `Metadata` each time; it does not cache. It also does not validate the token; an empty string produces `authorization: Bearer`, which the server will reject.

The interceptor

`PetServiceAuthInterceptor` is a `Grpc.Core.Interceptors.Interceptor` that wraps all five call shapes — unary, server-streaming, client-streaming, duplex and blocking unary — and attaches the contract's metadata:

```C#
var invoker = channel.Intercept(new PetServiceAuthInterceptor(() => tokenProvider.GetToken()));
var client = new PetService.PetServiceClient(invoker);
```

The constructor takes a `Func<string?>` rather than a string so a rotated token is picked up on the next call without rebuilding the interceptor. Returning `null` or `""` sends the call unauthenticated, which is how an anonymous operation is expressed.

The interceptor copies the caller's existing metadata first and then adds the scheme entries, so per-call metadata survives. It does not overwrite an `authorization` header the caller already set; it appends a second one, and servers typically reject duplicate authorization headers. If a call site supplies its own credential, do not also install the interceptor for that call.

The limits, stated plainly

- **One credential, every scheme.** The factory returns a single string, and the interceptor adds it under every scheme's key. A contract that combines a bearer token and an API key in one request cannot express two different values. The helpers can (one call each), the interceptor cannot.
- **Query API keys are misnamed in the helper.** For `in: query`, the interceptor uses the query parameter name as the metadata key, which is the only sensible choice; `CreateApiKeyMetadata` always emits `x-api-key` unless the scheme was declared as a header. Treat the helper as header-only.
- **Two schemes of one kind collide.** The helper name is fixed per kind, so two `apiKey` schemes (or two bearer schemes) both emit `CreateApiKeyMetadata` (or `CreateBearerTokenMetadata`) in the same class and the generated file does not compile.
- **Basic is not supported.** It becomes an `Unknown` scheme and a metadata entry keyed by the scheme name, which is not what an HTTP Basic server expects. Use a hand-written header.
- **Nothing is refreshed.** OAuth2 flows, token endpoints and scopes are documentation here. The application owns acquisition and renewal; the generator only formats what it is given.
- **Nothing is validated.** There is no server-side authentication filter or policy. The server reads `context.RequestHeaders` and decides (chapter 19).

For anything beyond a single bearer token or a single API key, the generated auth surface is a starting point, not a solution. The honest pattern is to disable it (`GenerateAuthInterceptors=false`) and write one interceptor that knows the contract's real credential rules.

22. The Gateway: gRPC in Front of REST

Proxy mode generates a `{Service}Proxy : {Service}Base` that answers gRPC calls by calling the legacy REST service the document describes. It is the only generated type that makes an HTTP request, and the only one whose behaviour depends on options read at run time rather than at build time.

Enabling it

`EnableProxyGeneration` (default `false`) adds one file value: the proxy class. It needs the `SharpPortico.Runtime` package, which supplies `ProxyOptions`, `IRestClient`, the cache, the key provider and the error mapper. Nothing is auto-registered; the host builds the object graph, as the sample does:

```C#
var options = new ProxyOptions
{
    BaseUrl = "http://localhost:5099",
    ApiKeyHeaderName = "X-Api-Key",
    CacheTtl = TimeSpan.FromSeconds(60),
    ClientKeyMode = ClientKeyMode.None
};

var rest = new HttpRestClient(httpClient, options.BaseUrl);
var proxy = new PetServiceProxy(options, rest,
    cache: cache, keys: keys, validator: validator, audit: audit);

var server = new Server { Services = { PetService.BindService(proxy) }, … };
```

`ProxyOptions.BaseUrl` is **not read by the generated proxy**. The address lives in the `HttpRestClient` constructor, which is why the sample passes the same string twice. Treat `BaseUrl` as documentation on the options object, not as the switch that moves the gateway.

The build-time proxy options follow the same rule. `EnableProxyGeneration` decides whether the proxy file is emitted; `ProxyBaseUrl`, `ProxyApiKeyHeaderName`, `ProxyCacheTtlSeconds`, `ProxyBypassCacheMetadataKey`, `ProxyClientKeyHeaderName`, `ProxyClientKeyMode` and `ProxyAuditEnabled` are accepted but not read by the emitter. The generated proxy takes the matching values from the `ProxyOptions` instance, and auditing happens when an `IProxyAuditLogger` is supplied (chapter 5).

`SharpPortico.Proxy.ProxyServiceCollectionExtensions.AddSharpPorticoProxyCore(options)` registers an `IHttpClientFactory`, an `IMemoryCache`, the options, a configuration-backed `IKeyProvider`, the default cache and the audit logger. It does not register the generated proxy or `IRestClient`; those are yours.

One call, step by step

For a unary RPC the generated override does the following, in order:

1. **Classifies the call.** `isRead` is true only when the original HTTP method was `GET`. Everything else is a write for caching purposes, including `HEAD`.
2. **Reads identity.** The client key is read from the gRPC metadata key named by `ClientKeyHeaderName` (`x-portico-key` by default), and the peer address from `ServerCallContext.Peer`.
3. **Chooses the outbound key.** `ClientKeyMode.None` uses `IKeyProvider`; `Forward` sends the inbound key as the outbound API key; `Own` validates the inbound key with `IClientKeyValidator` and then uses `IKeyProvider`. An invalid key in `Own` mode throws `Unauthenticated`.
4. **Builds the path.** The original OpenAPI path template is a literal in the generated code.
5. **Adds every set scalar request field** to both a path-parameter dictionary and a query-parameter dictionary.
6. **Serializes the body**, if the request has a `Body` message field, to UTF-8 JSON.

7. **Looks in the cache** for a read when a cache is installed and the bypass flag is absent.
8. **Sends** a `RestRequest(method, path, pathParams, queryParams, headers, body, "application/json")`.
9. **Maps a non-2xx status** to a gRPC status and throws `RpcException` with the response body as the message.
10. **Parses the 2xx body** into the response message, caches it if the call was a read, and returns.

The `headers` dictionary contains exactly one entry: the outbound API key, when there is one. No inbound gRPC metadata is forwarded.

What gets sent

The request the legacy service sees is not the request the OpenAPI document describes. Three rules explain almost all of the difference:

- **Every singular scalar field is a query parameter.** Path parameters are also added to the path, so a `GET /pets/{petId}` with `PetId = 7` becomes `/pets/7?PetId=7`. A server that ignores unknown query parameters is fine; a strict one is not.
- **Names are C# field names.** The query key is `PetId`, not `petId` or `pet_id`. The path substitution is case-insensitive, so `{petId}` is filled correctly, but the duplicate query entry carries the C# spelling.
- **Only a `Body` message field is a body.** A body that mapped to a scalar (`bytes body`) has no serialization path, and a non-optional `bytes` field also makes the file fail to compile (see below). A request with path and body fields sends the body as `application/json` regardless of the media type the contract declared.

Parameters that came from headers are query parameters here, because the parser does not record where a parameter came from (chapter 14) and the proxy has nothing to decide with. Message-valued fields are skipped entirely: they are neither path, query nor body unless the field is the body itself. Repeated scalar fields are not skipped, but the generated `IsSet` test compares the collection with `0`, so a proxy for such a request does not compile; chapter 23 records that failure.

What comes back

Any 2xx is success, whatever the document's success code was. The body is parsed as JSON even when the response `Content-Type` says otherwise. A 204 or an empty body leaves the response message at its defaults, which is why the empty-response `has_value` member exists.

A non-2xx becomes `RpcException` with a mapped code and the raw response body as `Detail`:

HTTP	gRPC
400, 422	`InvalidArgument` (3)
401, 403	`Unauthenticated` (16)
404	`NotFound` (5)
408, 504	`DeadlineExceeded` (4)
409	`AlreadyExists` (6)
429	`ResourceExhausted` (8)
500, 502	`Internal` (13)
503	`Unavailable` (14)
501, 505	`Unimplemented` (12)
anything else	`Unknown` (2)

`403` mapping to `Unauthenticated` rather than `PermissionDenied` is the mapper's choice, not gRPC's convention. A `405`, `410` or `418` is `Unknown`. The error body is plain text in `Detail`; a structured problem document is not parsed.

The cache

The cache is keyed by `{Service}/{Rpc}/{path}` followed by every query entry in insertion order. It stores the raw response bytes and returns them without calling the legacy service. Only `GET`-shaped RPCs are cached; the `CacheReadsOnly` option is declared but not consulted, so setting it to `false` does not enable write caching.

A call bypasses the cache when the gRPC metadata entry named by `BypassCacheMetadataKey` (`x-portico-bypass-cache` by default) is `true`. That is the per-call escape hatch:

```C#
var headers = new Metadata { { "x-portico-bypass-cache", "true" } };
await client.GetPetAsync(new GetPetRequest { PetId = 1 }, headers);
```

There is no invalidation on writes, no variation by client identity, no `Vary` handling and no negative caching. A write through the gateway leaves every cached read stale until its TTL expires. For a contract with more than trivial write traffic, install `NullProxyCache` or keep the TTL very short.

What the gateway cannot do

- **Streaming.** Only unary RPCs are overridden. A server-streaming RPC falls back to the base implementation, which completes an empty stream; client- and bidi-streaming RPCs throw `Unimplemented`. Because pagination can turn a list operation into server streaming (chapter 13), a paginated contract may produce a gateway method that silently returns nothing.
- **Non-JSON payloads.** `application/octet-stream`, `multipart/form-data` and form-encoded bodies are not handled. Binary request fields are skipped, binary response fields are not parsed.
- **Header parameters.** They become query parameters. An API that requires `X-Tenant` as a header will not see it.
- **Cancellation and deadlines.** `HttpRestClient.Send` is synchronous and passes `CancellationToken.None`. A client that cancels a gRPC call does not cancel the outbound HTTP request; the `HttpClient.Timeout` is the only limit.
- **Free-form and binary fields.** `Struct`, `Value` and non-optional `bytes` fields make the generated proxy file fail to compile, for the reasons in chapters 15 and 16.

These are not configuration gaps. They are the edges of what a proxy generated from a lossy contract can know. Chapter 23 is the JSON mapping in detail, including the one defect that affects otherwise supported contracts.

23. JSON at the Gateway

The proxy does not use `System.Text.Json`'s serializer or protobuf's `JsonFormatter`. It emits a serializer and a parser per message, as C# code, because the generated messages have no descriptor and the project may be AOT-published. This chapter is the exact grammar of what those helpers read and write.

Writing a request body

`SerializePet(msg)` is a `StringBuilder` walk over the message's fields. For each field it appends `"proto_name":value`, in declaration order, with no whitespace:

```json
{"id":7,"name":"Rex","status":2}
```

Field	Written as
string	the string between quotes, **without escaping**
bool	`true` / `false`
int32, int64, uint32, uint64, float, double	invariant-culture number
enum	its number
bytes	a base64 string
message	a nested object, or `null`
repeated	a JSON array; messages recurse, scalars use the same rules
proto3 `optional`, set	its value
proto3 `optional`, not set	`null`

Four decisions in that table are worth spelling out.

Keys are proto names, not OpenAPI names. The key is `pet_id`, not `petId`. The descriptor-side name comes from the message field (chapter 8), so a legacy API that expects camelCase sees different JSON from the document it published. If the REST service is strict about property names, the gateway is not a drop-in replacement.

Every non-optional field is written, including defaults. An unset `int` is `0`, an unset `string` is `""`, an unset list is `[]`, an unset message is `null`. This is not proto3 JSON, which omits defaults. A legacy API that distinguishes "absent" from "zero" will see a zero the caller never sent — unless the contract marked the field nullable, in which case the field is `optional` and unwritten means `null`.

Enums are numbers and 64-bit integers are numbers. Proto3 JSON spells both as strings. The proxy's grammar is closer to a typical REST API than to protojson, and a consumer must not assume the two are interchangeable.

Strings are not escaped. `sb.Append('"').Append(value).Append('"')` is the whole implementation. A value containing `"`, `\`, a newline or a control character produces invalid JSON, and the legacy service rejects the request or misparses it. This is a real defect in 1.5.0, not a documented trade-off; a contract whose string fields can contain quotes needs a hand-written adapter. The same applies to `NaN` and `Infinity`, which are written as bare tokens that JSON does not allow.

Reading a response

`ParsePet(byte[] body, Pet msg)` parses the body with `JsonDocument` and walks the same field list. For each field it looks up the **exact** proto-name key:

```C#
if (el.ValueKind == JsonValueKind.Object && el.TryGetProperty("status", out var _status))
{
    msg.Status = (StatusEnum)_status.GetInt32();
}
```

JSON	Read as
missing key	the field's default, untouched
extra key	ignored
`null` on a non-optional field	strings become `""`, messages stay unset; a numeric, boolean or enum `null` throws `InvalidOperationException` from `Get…`
`null` on an `optional` field	the field stays absent (`HasX` false)
number	`GetInt32` / `GetInt64` / `GetUInt32` / …
string	`GetString` , or `GetBytesFromBase64` for `bytes`
number for an enum	cast from `GetInt32`
string for an enum	**throws** — there is no name lookup
array	read only when the field is repeated

Keys are case-sensitive and are not tried in camelCase. A response that sends `petId` or `PetId` does not fill a field whose proto name is `pet_id` . A response that sends an enum as `"available"` instead of `0` throws `InvalidOperationException` , which escapes the proxy handler and becomes an `Internal` gRPC error. A malformed document throws `JsonException` the same way.

Wrapper unwrapping

A REST response is often just the payload, while the gRPC response is a wrapper message with one `data` or `items` field. The parser handles that with a root fallback: if the wrapper has exactly one non-sentinel field, and the JSON root does not contain its key, the root is parsed as that field's type.

For `GetPetResponse { Pet data = 1; }` and a body of `{"id":7,"name":"Rex"}` :

- `data` is absent, so the whole object is parsed as a `Pet` ;
- a body of `{"data":{…}}` still works, because the key is found first.

For `ListPetsResponse { repeated Pet items = 1; }` and a body of `[{"id":7}]` , the wrapper's single field is a repeated message, so the root array is read as `items` . This fallback exists only at the top level and only for a wrapper with exactly one field; a nested message's parser does not try it.

What the helper cannot represent

The serializer and parser are generated per message from the field kinds, and four field shapes have no generated path:

Shape	Result
`google.protobuf.Struct` / `Value` anywhere in a serialized message	`SerializeStruct` / `ParseStruct` are referenced but never emitted: **CS0103**
non-optional `bytes` at the top level of a request	`IsSet` compares a `ByteString` with `0` : **CS0019**
repeated scalar at the top level of a request	`IsSet` reads `.Length` or compares a collection with `0` : **CS0019/CS1061**
any message-typed request field other than `Body`	silently omitted from path, query and body

A `Struct` in a response breaks the compiler in the same way; the failure is in the parse helper, not in the descriptor. These are the reasons a contract that maps cleanly to protobuf can still produce a proxy that does not build. The generated messages, server and client are unaffected — only the gateway file is.

The supported subset is therefore: unary RPCs, singular scalars and enums as parameters, a single message `Body`, JSON bodies, and responses whose shapes are made of scalars, enums, nested messages and repeated messages. That subset covers a large share of ordinary REST APIs, and it is worth checking against a contract before enabling `EnableProxyGeneration`.

The string-escaping defect

The one runtime defect that affects an otherwise supported contract is the writer's treatment of strings. It concatenates the value between quotes without escaping, so a name containing a double quote produces a body the legacy service cannot parse. The parser is not affected: it uses `JsonDocument`, which handles escapes correctly. Until the writer is fixed, a proxy in front of an API whose string fields can contain quotes, backslashes or newlines needs a hand-written serializer or a contract that encodes those values (base64, for example).

Nothing here is proto3 JSON. The gateway's grammar is its own, generated from the message model, and a caller who assumes protojson's rules — string-encoded 64-bit integers, enum names, omitted defaults — will be surprised in both directions.

24. The Command Line

`sharpportico` has one command. It runs the same parser and the same `ProtoEmitter` the build runs, prints what the mapping decided, and optionally writes the descriptor to a file. It does not generate C#.

Usage

```text
usage: sharpportico generate <openapi.yaml|json> [--out DIR|FILE] [--service-name NAME]
                             [--namespace NS] [--format text|json]
```

The first argument must be `generate`; `--help` and `-h` print the same text and exit 0. Options are separate arguments (`--out build`), not `--out=build`; a value glued to the name is reported as an unknown argument and exits 2.

Option	Effect
`--out DIR`	writes `<spec>.proto` inside `DIR`, creating it if needed
`--out FILE.proto`	writes exactly that file; the `.proto` suffix is the switch
`--service-name NAME`	service base name; `Service` is appended unless the name already ends with it
`--namespace NS`	namespace of the generated C#; defaults to the service name
`--format text`	the default: a summary on stdout, diagnostics on stderr
`--format json`	one JSON document on stdout and nothing else

Without `--service-name`, the file stem is used, so `sink.yaml` becomes `SinkService` after the suffix is applied. Without `--namespace`, the namespace equals the service name. Both are the defaults a build applies through `AdditionalFileRequest.ToWorkItem`, not a second set of rules maintained by the CLI.

A text run

Running the command against the manual's `sink.yaml`:

```text
spec:       docs/manual/fixtures/sink.yaml
service:    SinkService
namespace:  SinkService
package:    sinkservice
rpc:        SinkService (4)
  ListWidgets - GET /widgets (Unary)
  CreateWidget - POST /widgets (Unary)
  DeleteWidget - DELETE /widgets/{id} (Unary)
  UploadPhoto - PUT /widgets/{id}/photo (Unary)
messages:   10
enums:      2
```

The `rpc:` line gives the count; each following line is one operation, with the RPC name, the original HTTP method and path, and the kind the mapping chose. Those four columns are the entire contract-to-RPC decision in one place, which makes the command useful in review even when no file is written.

The package is the lower-cased service name because no package option was supplied. `--service-name` does not change the package independently; the package follows the same derivation a build uses (chapter 8).

A JSON run

```text
sharpportico generate docs/manual/fixtures/sink.yaml --format json --out build
```

writes one document to stdout and the descriptor to `build/sink.proto`:

```json
{
  "formatVersion": 1,
  "spec": "docs/manual/fixtures/sink.yaml",
  "service": {
    "name": "SinkService",
    "namespace": "SinkService",
    "package": "sinkservice"
  },
  "services": [
    {
      "name": "SinkService",
      "rpcs": [
        {
          "name": "ListWidgets",
          "httpMethod": "GET",
          "path": "/widgets",
          "kind": "Unary"
        },
        {
          "name": "CreateWidget",
          "httpMethod": "POST",
          "path": "/widgets",
          "kind": "Unary"
        },
        {
          "name": "DeleteWidget",
          "httpMethod": "DELETE",
          "path": "/widgets/{id}",
          "kind": "Unary"
        },
        {
          "name": "UploadPhoto",
          "httpMethod": "PUT",
          "path": "/widgets/{id}/photo",
          "kind": "Unary"
        }
      ]
    }
  ],
  "counts": {
    "messages": 10,
    "enums": 2
  },
  "diagnostics": [],
  "descriptor": {
    "path": "build/sink.proto"
  }
}
```

The writer emits members explicitly rather than serializing a type, so the document is stable and reflection-free: two runs over the same input produce the same bytes, with two-space indentation, `\n` line endings and one trailing newline. `formatVersion` is the document's version (`1`), independent of the package version.

`diagnostics` carries the same id, severity, message and line a build would report. A refused contract has no `service`, `services` or `descriptor`, but it still has diagnostics — that is what a gate reads.

Exit codes and streams

Code	Meaning
0	the contract mapped; the summary or JSON was written
1	the contract was refused, or the run threw; diagnostics explain why
2	a usage error: no command, unknown command, missing spec, unknown argument, bad `--format`, file not found

The split between streams is deliberate:

- in **text** mode, the summary goes to stdout and diagnostics to stderr. Redirecting stdout to a file keeps the diagnostics visible on the console, which is what a developer wants;
- in **json** mode, the report goes to stdout as one document. A refused contract still produces that document, so a CI step can parse stdout without stripping warning lines; only an unexpected exception reports on stderr.

A warning does not change the exit code. A contract with `SP1003` or `SP2000` warnings maps and exits 0; the warnings are in the report for a gate that chooses to fail on them. Only an error diagnostic, or a refusal with no model, exits 1.

What the command cannot do

- **It does not write C#.** The descriptor and the report are the outputs; message classes, the service and the proxy come from the source generator during a build.
- **It has no `validate` or `mapping` subcommand.** `generate` is the only command. A validation-only run is `generate` without `--out`: it maps, reports and writes nothing.
- **It takes one file.** There is no glob or directory mode; run it once per document in a loop.
- **It exposes no mapping options.** Pagination names, streaming hints, proxy switches and the rest are fixed at their build defaults, because the CLI builds the work item through the same `AdditionalFileRequest` path. A contract that depends on a non-default option cannot be reproduced exactly from the command line.
- **It does not read stdin.** The first argument is a path that must exist.

The last two limitations are the ones to remember when the CLI disagrees with a build. If a build sets `DetectPagination=false` or `RespectStreamingHints=false`, the CLI still applies the default, and the RPC kinds in its report will not match the generated code. The descriptor text itself is produced by the same emitter, so for the default configuration the file and the `{Service}Proto.Text` constant are byte-for-byte equal.

Using it in CI

A contract gate is three lines:

```shell
sharpportico generate openapi/petstore.yaml --format json > mapping.json
jq -e '.diagnostics | map(select(.severity == "error")) | length == 0' mapping.json
```

The second line fails the job on an error without depending on the exit code; the first already exits 1 on a refusal. For a descriptor diff between two revisions, write the `.proto` with `--out` and compare the files: the output is LF-normalized regardless of the host, so a Windows and a Linux job produce identical bytes for the same input.

The next part turns to the diagnostics those jobs read: every `SP` id, what produces it, and what to do about it.

25. Diagnostics

SharpPortico reports what it could not do with `SP` ids, through the compiler and through the CLI, with the same text in both. The numbering is by stage: `SP1xxx` while reading the document, `SP2xxx` while mapping it, `SP3xxx` while emitting code.

Two channels, one message

Channel	Where it appears
source generator	a warning or error on the `AdditionalFiles` item, in the build log and the IDE error list
CLI	stderr in `--format text`, the `diagnostics` array in `--format json`

Both carry the same id, severity and message because both call the same `DiagnosticDescriptor`. A suppression in `.editorconfig` therefore works for a build, and a saved JSON report can be compared with a later one without normalising the text.

Severity means one thing

Severity	Meaning	What was emitted
`error`	the contract cannot become code that compiles	nothing for that specification
`warning`	the contract mapped; something did not survive	the code, with the loss named

An error is a refusal, not a partial result. Emitting the valid half would give a consumer a service missing an operation, or a message shadowing another, and the build would be green while the contract was wrong.

`SP1000` is a warning by severity even though it produces no code: the document could not be read, so there is nothing to map. A build reports the warning and compiles no generated types. When the CLI can open the path but the document does not map, it exits 1 because there is no model; a path that does not exist at all is a usage error and exits 2 (chapter 24).

The ids

Id	Severity	Raised when
`SP1000`	warning	the file is missing, empty, or not valid YAML/JSON; the reader's own complaint is appended
`SP1002`	warning	a 3.1 document was rewritten to 3.0 before parsing
`SP1003`	warning	one 3.1 construct was mapped with a loss
`SP2000`	warning	a nullable array or nullable element could not keep its nullability
`SP2005`	**error**	two messages would be emitted with one name
`SP2006`	**error**	a property's member name equals its enclosing type's name
`SP2007`	**error**	two inline enumerations would share a name with different members
`SP2008`	**error**	a nullable property's `Has` / `Clear` member collides with another member

Six more ids are declared and never raised by 1.5.0: `SP1001`, `SP2001`, `SP2002`, `SP2003`, `SP2004` and `SP3000`. They are described at the end of this chapter, because a published id keeps its meaning even when a version has no path to it.

Parse: SP1000

```
TEXT

warning SP1000: SharpPortico could not parse OpenAPI file 'openapi/pets.yaml': ...
```

The path is resolved against the project directory for a build, and used exactly as given by the CLI. The suffix is the OpenAPI reader's own message, which is why it is not paraphrased: it names the line and the token that failed. Common causes are a YAML tab where indentation is required, an unquoted `:` in a description, a `$ref` that points at a missing file, or a JSON document with a trailing comma.

`SP1000` is also what a document that is valid YAML but not an OpenAPI 3.x document produces, because the bundled reader refuses it before the generator sees a version (see `SP1001` below).

Parse: SP1002 and SP1003

`SP1002` is informational. It appears once for every 3.1 document and says how many constructs were rewritten:

```
TEXT

warning SP1002: SharpPortico parsed 'openapi/fleet-3.1.yaml' as OpenAPI 3.0: the bundled parser
(Microsoft.OpenApi 1.6.x) does not read a 3.1 document, so it was rewritten to the 3.0 equivalent
first - the declared version, $defs references, webhooks, a type array, const, prefixItems,
contentEncoding, a numeric exclusive bound
```

`SP1003` appears once per construct that the rewrite mapped without carrying everything:

```
TEXT

warning SP1003: SharpPortico mapped the tuple at components/schemas/Vehicle/properties/bounds in
'openapi/fleet-3.1.yaml', but a JSON Schema tuple's fixed length and per-position types have no
protobuf equivalent, so it maps to a repeated field
```

The second argument is the location inside the document, not a line number: a 3.1 document is rewritten as a tree, so the source line of a rewritten node is not reliably known and the generator does not guess. Chapter 6 lists which constructs produce which kind of loss; chapter 17 says what each loss costs.

Mapping warnings: SP2000

`SP2000` is the only warning the mapper raises about a construct it still mapped:

```
TEXT

warning SP2000: SharpPortico could not map 'nullable' (tags): a repeated field has no per-element
presence, so an element that was null reads back as the element's default value
```

The three arguments are the construct (`nullable`), the location (`Message/field` , or `field[]` for the element), and what was lost. Chapter 12 explains why a repeated field cannot carry presence, and chapter 10 gives the workaround: a wrapper message with a presence field per element.

This warning is gated by `EmitDiagnosticsForUnmappableConstructs` (default `true`). Turning it off removes the warning and changes nothing else; the mapping is the same. It is the one diagnostic a consumer may legitimately silence, which is why it has a switch and the errors do not.

Mapping errors: SP2005–SP2008

An error means the generated file would not compile, so the generator refuses the whole document. Each message names the two things that collide and asks for a rename; none of them attempts a rename of its own, because a generated name that differs from the contract is harder to trace than a build failure.

SP2005 — two messages, one name

```
TEXT

error SP2005: SharpPortico cannot map 'openapi/pets.yaml': the message name 'GetPetRequest' is
produced twice - an operation derives its messages as '{OperationId}Request' and
'{OperationId}Response', so a component schema of that name collides. Rename one of the two
```

An operation `getPet` derives `GetPetRequest` and `GetPetResponse`. A component schema named `GetPetRequest` produces the same message name. Rename the schema (or the operation id). The same collision can happen between two component names that sanitise to the same PascalCase string, for example `pet-details` and `PetDetails` (chapter 8).

SP2006 — a member named like its type

```
TEXT

error SP2006: SharpPortico cannot map 'openapi/things.yaml': property 'thing' of schema 'Thing'
would generate a member with the same name as its enclosing type, which C# forbids (CS0542).
Rename the property or the schema
```

C# forbids a member with its enclosing type's name. JSON Schema allows it, and a contract written without C# in mind often contains it. Renaming either side is the only fix; the generator will not suffix the property.

SP2007 — two enumerations, one name

```
TEXT

error SP2007: SharpPortico cannot map 'openapi/things.yaml': the inline enumeration named
'StatusEnum' is declared twice with different members ('available' and 'pending'). An inline
enumeration is named after the property declaring it, so two schemas with a property of that name
collide - declare one of them as a named schema under components
```

An inline enum is named `<Property>Enum`, so two schemas with a `status` property and different members both want `StatusEnum`. The mapper first derives a distinct name from the original plus the declared members (`StatusEnum` + `SoldReturned`); if that name is free, both survive and no error is raised. `SP2007` appears only when the derived name is **also** taken by a different member set, at which point no name is left that the contract actually implies. Promote one to `components.schemas` to choose its name yourself. If the two member sets are identical, the mapper reuses one enum and no error is raised.

SP2008 — presence collides with a property

```
TEXT

error SP2008: SharpPortico cannot map 'openapi/things.yaml': property 'more' of schema 'Thing' is
nullable, so its message needs presence members named 'HasMore' and 'ClearMore', but 'HasMore' is
already a member of 'Thing'. Rename one of the two
```

A nullable `more` becomes `optional` and gains `HasMore` / `ClearMore`. A sibling property `hasMore` sanitises to `HasMore`, and the two cannot coexist. Rename the nullable property or the property that spells its presence member.

Ids that 1.5.0 does not raise

Six declared ids have no code path:

Id	Declared as	Why nothing reaches it
`SP1001`	document is not OpenAPI 3.x	the reader refuses it first, and its complaint is reported as `SP1000`
`SP2001`	unsupported HTTP method	every verb on a path item becomes an RPC, so no operation is dropped for its method
`SP2002`	unsupported schema composition	accepted compositions are resolved; 3.1 rewrites report `SP1003`
`SP2003`	circular reference	a self-referencing message is legal protobuf, so there is no cycle to report

Id	Declared as	Why nothing reaches it
`SP2004`	unsupported auth scheme	an unmodelled scheme still gets a metadata helper keyed by its name
`SP3000`	emit failure	emission is not wrapped in a handler, so a failure surfaces as `AD0001`

They stay in the registry because an id that has appeared in a build log, a saved report or a suppression must keep meaning the same thing. A manual that reused the numbers for new conditions would make old logs lie.

Gating a build

Errors already fail a build. A warning is a warning until the project says otherwise, and MSBuild promotes by id:

MSBUILD / XML

```
<PropertyGroup>
  <!-- This repository's contracts are not allowed to lose nullability quietly. -->
  <WarningsAsErrors>$(WarningsAsErrors);SP2000</WarningsAsErrors>
</PropertyGroup>
```

For a compiler-free gate, the CLI is enough: a refusal exits 1, and the JSON report carries the severity of every diagnostic for a `jq` filter (chapter 24). Warning ids can be promoted the same way in CI by filtering the report; suppressing an id in `.editorconfig` works for the build channel only, because the CLI does not read editor configuration.

26. Symptom Index

This chapter starts from what you see — a compiler error, an empty stream, a field that never arrives — and works back to the rule that produced it. Every entry names the chapter that explains the cause in full.

Build symptoms

Symptom	Most likely cause	Chapter
No generated types at all; the namespace does not exist	the spec is not an `AdditionalFiles` item, or the attribute names a path that does not exist	4
`warning SP1000` and no generated types	the file is missing, empty or invalid YAML/JSON; the warning carries the reader's message	25
`error SP2005`	a component schema collides with an operation-derived `{OperationId}Request` / `Response`	25
`error SP2006` / `CS0542`	a property has the same name as its enclosing message	25
`error SP2007`	two inline enums with different members land on the same derived name	25
`error SP2008`	a nullable property's `Has` / `Clear` member collides with a sibling property	25
`warning SP1002`	a 3.1 document was rewritten; this is expected	6
`warning SP1003`	a 3.1 construct was mapped with a loss; the message names it	6
`warning SP2000`	a nullable array or element cannot carry its nullability	12
`CS0111` in `{Service}Auth`	two security schemes of one kind emit the same helper name	21
`CS0103: SerializeStruct` / `ParseStruct`	the contract has a `Struct` or `Value` field and proxy generation is on	23
`CS0019` / `CS1061` in `{Service}Proxy`	a `bytes` or repeated scalar at the top level of a request	23
`CS0246: google.protobuf.Struct`	the generated message references a well-known type but `Google.Protobuf` is not referenced	3
`CS0246: {Service}Base`	`EmitServer=false`, but the contract still emits both `BindService` overloads	3
`AD0001` from the generator	an emit-stage failure; `SP3000` is declared but not raised in 1.5.0	25

The first row is worth checking before anything else. A source generator runs only over files the project gives it: an `AdditionalFiles` item in the `.csproj`, or an `[assembly: OpenApiToGrpc("path")]` attribute. A file that is merely present in the repository is not an input.

Runtime symptoms

Symptom	Most likely cause	Chapter
`Unimplemented` from a unary call	the server implementation did not override `XAsync`	19
`Unimplemented` from a streaming call	a client- or bidi-streaming RPC was not overridden	19
A server-streaming call returns zero messages	the base default completes the stream; override `X`	19
`Unimplemented` from the proxy	the RPC is streaming; the proxy only overrides unary	22
An empty stream from the proxy	pagination made the RPC server-streaming, and the base default completes it	22
`Internal` with a JSON message	the proxy could not parse the response (enum as string, malformed JSON)	23
`Unknown` with an HTTP body	the legacy status did not map to a specific gRPC code	22

Symptom	Most likely cause	Chapter
Stale data through the proxy	the GET response is cached for `CacheTtl`; send the bypass metadata	22
`Cancelled` instead of `OperationCanceledException`	gRPC reports cancellation as an `RpcException`	20
`ResourceExhausted` on a large message	the channel or server message-size limit, not the contract	20
A body field missing at the legacy service	the proxy only sends a message field named `Body`, as JSON	22
A query parameter has the wrong name	the proxy sends the C# field name (`PetId`), not the OpenAPI name (`petId`)	22
`CS0246` for an inline object type (for example `Dims`)	1.5.0 names inline object messages but does not emit them	declare a named component and use `$ref`
A response property never fills	the JSON key is not the exact proto name (`pet_id`)	23

Two symptoms that look like generator bugs

"The field is always zero." Check the descriptor before the code. A field number that changed between the client's generated code and the server's is a wire mismatch: both sides read a valid message, and each puts the value in a different field. Regenerate both from the same document and the same version. This is why chapter 14 treats parameter order as a wire contract: appending is safe, reordering is not.

"The service is there but grpcurl cannot see it." The generated messages return `null` from `Descriptor` (chapter 18), so gRPC server reflection has nothing to enumerate. Pass the `.proto` file to `grpcurl` explicitly; the descriptor is available as `{Service}Proto.Text` and from `sharpportico generate --out`.

A triage order

When the cause is not obvious, four steps find it faster than reading the generated file from the top.

1. **Read the first SharpPortico diagnostic, not the first C# error.** A refusal produces no code, and the C# errors that follow are consequences of a missing type. `SP2005` – `SP2008` explain the refusal; a C# error usually does not.
2. **Run the CLI on the same file.** If the CLI maps the contract and the build does not, the difference is an option: the build may set `RespectStreamingHints=false`, change the pagination names, or use a different service name. The CLI always uses the defaults (chapter 24).
3. **Look at the generated file.** Set `EmitCompilerGeneratedFiles` and read `obj/.../generated/SharpPortico.SourceGenerator/...`:

```
xml <PropertyGroup> <EmitCompilerGeneratedFiles>true</EmitCompilerGeneratedFiles>
<CompilerGeneratedFilesOutputPath>$(BaseIntermediateOutputPath)generated</CompilerGeneratedFilesOutputPath>
</PropertyGroup>
```

The `.g.cs` is the code that compiles, and the `.proto.cs` is the descriptor. A type that is missing from the first is not a C# problem. 4. **Compile the descriptor.** Write it with `sharpportico generate --out` and run `protoc --descriptor_set_out`. A descriptor that `protoc` rejects has a mapping-level defect (an enum without a zero member, a duplicate field number) that the C# compiler cannot see.

What to put in a report

If the cause is a generator defect rather than a contract problem, the smallest useful report contains:

- the document, or the smallest excerpt that still reproduces it;
- the version from `{Service}Proto.Text`'s generated-code attribute, or `dotnet list package`;

- the build options, because they change the mapping;
- the full text of the first `SP` diagnostic, if there is one;
- whether the failure is a refusal (no generated code) or a compile error in generated code.

The distinction in the last point decides which part of the generator is at fault: a refusal comes from the parser or mapper, a compile error from an emitter. The two are fixed in different places, and a report that conflates them sends the reader to the wrong file.

When the answer is "that is the mapping"

Some symptoms are the documented behaviour and will not change:

- a `oneOf` loses its alternatives (chapter 9);
- a non-success response has no message (chapter 17);
- a `Struct` field breaks the proxy (chapter 23);
- the proxy sends query names in C# spelling (chapter 22);
- a repeated field cannot carry nullability (chapter 12).

For those, the fix is in the contract or in a hand-written adapter, not in a bug report. The rules are in Part III precisely so that this answer can be given with a chapter number instead of a shrug.

27. The Generated Surface, Member by Member

This is the reference page: what exists in the namespace after a build, what each member is for, and which option switches it off. Signatures are shown as they are emitted, with the service name from `petstore.yaml` (`PetstoreService`) as the example.

Messages and enums

Member	Shape	Option
one class per message	`public sealed partial class Pet : IMessage<Pet>`	always
one property per field	scalar, message, `RepeatedField<T>`, or presence-backed `optional`	always
`Parser`, `Descriptor`, `Clone`, `Equals`, `GetHashCode`, `ToString`, `WriteTo`, `CalculateSize`, `MergeFrom`, `IsInitialized`	`IMessage<T>` implementation	always
one enum per enumeration	`public enum StatusEnum { Available = 0, … }`	always
descriptor constant	`public static partial class PetstoreServiceProto { public const string Text = "…"; }`	`EmitProtoFile`

`Descriptor` returns `null!`; chapter 18 explains what that costs. The descriptor text is LF-normalized and is the same string `sharpportico generate --out` writes.

The contract class

```C#
public static partial class PetstoreService
{
    public const string ServiceFullName = "petstoreservice.PetstoreService";

    public static readonly Method<ListPetsRequest, ListPetsResponse> Method_ListPets;
    public static readonly Method<CreatePetRequest, CreatePetResponse> Method_CreatePet;
    public static readonly Method<GetPetRequest, GetPetResponse> Method_GetPet;

    public static ServerServiceDefinition BindService(PetstoreServiceBase serviceBase);
    public static void BindService(ServiceBinderBase binder, PetstoreServiceBase serviceImpl);

    public partial class PetstoreServiceClient : ClientBase<PetstoreServiceClient> { … }
}
```

The nested client is what `protoc` would call the stub. Its constructors are `internal` / `protected`; application code uses the convenience client.

The server base

```C#
[BindServiceMethod(typeof(PetstoreService), "BindService")]
public abstract partial class PetstoreServiceBase
{
    public virtual Task<ListPetsResponse> ListPetsAsync(ListPetsRequest request, ServerCallContext context);
    public virtual Task<CreatePetResponse> CreatePetAsync(CreatePetRequest request, ServerCallContext context);
    public virtual Task<GetPetResponse> GetPetAsync(GetPetRequest request, ServerCallContext context);
}
```

Streaming kinds replace the unary shape with the matching `IServerStreamWriter` / `IAsyncStreamReader` parameters (chapter 19). The unary shape also emits `XHandler` and `X` forwarding members; override `XAsync`.

The convenience client

```C#
public sealed partial class PetstoreServiceClient : PetstoreService.PetstoreServiceClient
{
    public static PetstoreServiceClient Create(GrpcChannel channel);
    public static PetstoreServiceClient Create(ChannelBase channel);       // throws unless GrpcChannel

    public Task<ListPetsResponse> ListPetsAsync(ListPetsRequest request, Metadata? headers = null,
        CancellationToken ct = default);
    public Task<CreatePetResponse> CreatePetAsync(CreatePetRequest request, Metadata? headers = null,
        CancellationToken ct = default);
    public Task<GetPetResponse> GetPetAsync(GetPetRequest request, Metadata? headers = null,
        CancellationToken ct = default);
    public Task<GetPetResponse> GetPetAsync(long petId, CancellationToken ct = default);
    public Task<Pet> GetPetDataAsync(GetPetRequest request, CancellationToken ct = default);
    public Task<Pet> GetPetDataAsync(long petId, CancellationToken ct = default);
}
```

The request-scalar and response-field overloads appear only when the request or response has exactly one field (chapter 20). A streaming RPC gets `XStream(…)` returning the matching call object.

Dependency injection

```C#
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddSharpPorticoPetstoreService(this IServiceCollection services);
}
```

It registers one singleton `PetstoreServiceClient` built from a `GrpcChannel` already in the container. It does not register a channel, a server or a proxy.

Authentication

```C#
public static partial class PetstoreServiceAuth
{
    public static Metadata CreateBearerTokenMetadata(string token);
    public static Metadata CreateApiKeyMetadata(string key);
}

public sealed class PetstoreServiceAuthInterceptor : Interceptor
{
    public PetstoreServiceAuthInterceptor(Func<string?> credentialFactory);
}
```

Both are emitted only when the document declares a security scheme and the corresponding option is on. The interceptor attaches the credential to every call shape; the helpers are per-call (chapter 21).

The proxy

```C#
public sealed partial class PetstoreServiceProxy : PetstoreServiceBase
{
    public PetstoreServiceProxy(
        ProxyOptions options,
        IRestClient rest,
        IProxyCache? cache = null,
        IKeyProvider? keys = null,
        IClientKeyValidator? validator = null,
        IProxyAuditLogger? audit = null);
}
```

It overrides unary RPCs only. `EnableProxyGeneration` (default `false`) emits it, and the project must reference `SharpPortico.Runtime` (chapters 22–23).

Options that change the surface

Option	Default	Effect on names or members
`EmitProtoFile`	`true`	removes `{Service}Proto`
`EmitClient`	`true`	removes the convenience client; set `EmitDependencyInjection=false` too, because the DI extension constructs that client and will not compile without it
`EmitServer`	`true`	removes `{Service}Base`; the two `BindService` overloads remain and reference it, so the file does not compile with this switch off alone
`EmitDependencyInjection`	`true`	removes `ServiceCollectionExtensions`
`GenerateAuthMetadataHelpers`	`true`	removes `{Service}Auth`
`GenerateAuthInterceptors`	`true`	removes `{Service}AuthInterceptor`
`EnableProxyGeneration`	`false`	adds `{Service}Proxy`
`ServiceNameSuffix`	`Service`	changes every name in this chapter
`Namespace` / `ProtoPackage`	derived	changes the C# namespace and the proto package

28. A Worked Example End to End

This chapter follows one small contract from YAML to a running call, naming the rule behind every line. The document is `fixtures/petstore.yaml`; the descriptor is what 1.5.0 produced from it.

The document

YAML

```yaml
openapi: "3.0.3"
info: { title: Petstore, version: "1.0.0" }
paths:
  /pets:
    get:
      operationId: listPets
      parameters:
        - { name: limit, in: query, schema: { type: integer } }
        - { name: page,  in: query, schema: { type: integer } }
      responses:
        "200":
          content:
            application/json:
              schema:
                type: array
                items: { $ref: "#/components/schemas/Pet" }
    post:
      operationId: createPet
      requestBody:
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Pet" }
      responses:
        "201":
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Pet" }
  /pets/{petId}:
    get:
      operationId: getPet
      parameters:
        - { name: petId, in: path, required: true, schema: { type: integer, format: int64 } }
      responses:
        "200":
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Pet" }
components:
  schemas:
    Pet:
      type: object
      properties:
        id:     { type: integer, format: int64 }
        name:   { type: string }
        status: { type: string, enum: [available, pending, sold] }
```

Nothing here is unusual, which is the point: the rules have to work on the ordinary case before the exotic ones matter.

What each rule does

Operations become RPCs. Three operations, three unary RPCs. The names are `SanitizePascal(operationId)`: `listPets` → `ListPets`, `createPet` → `CreatePet`, `getPet` → `GetPet` (chapter 13). The methods keep their HTTP verbs only in the report, not in the descriptor.

Parameters become fields in declaration order. `limit` is field 1, `page` is field 2. Their order is the wire contract; a later revision may append a parameter, not insert one (chapter 14).

The body comes last. `CreatePetRequest` has one field, `body = 1`, because there are no parameters (chapter 14). Its type is `Pet`, because the media type's schema is a `$ref`.

The array response becomes a wrapper with `items`. A `200` whose schema is an array produces `ListPetsResponse { repeated Pet items = 1; }`. The element is a `$ref`, so no inline element message is created (chapter 10).

The object response becomes `data`. A response whose schema is a `$ref` produces `{ Pet data = 1; }`, not a bare `Pet` return. Every RPC returns a message (chapter 13).

The inline enum becomes `StatusEnum`. The property `status` declares the members; the enum is named after the property plus `Enum`, and members are sanitised to PascalCase. `available` is 0, so the descriptor is valid proto3 (chapter 11).

`int64` stays `long`. A `format: int64` is not turned into a string. The JSON gateway serializes it as a number regardless of protojson's convention (chapters 7 and 23).

`201` is the success response. It is the only 2xx, so it is the one mapped. A `default` or a `4xx` would not add a field (chapter 17).

The descriptor

PROTO

```
syntax = "proto3";

package petstoreservice;

option csharp_namespace = "PetstoreService";

enum StatusEnum
{
    AVAILABLE = 0;
    PENDING = 1;
    SOLD = 2;
}

message Pet
{
    int64 id = 1;
    string name = 2;
    StatusEnum status = 3;
}

message ListPetsRequest
{
    int32 limit = 1;
    int32 page = 2;
}

message ListPetsResponse
{
    repeated Pet items = 1;
}

message CreatePetRequest
{
    Pet body = 1;
}

message CreatePetResponse
{
    Pet data = 1;
}

message GetPetRequest
```

```
{
    int64 pet_id = 1;
}

message GetPetResponse
{
    Pet data = 1;
}

service PetstoreService
{
    rpc ListPets (ListPetsRequest) returns (ListPetsResponse);
    rpc CreatePet (CreatePetRequest) returns (CreatePetResponse);
    rpc GetPet (GetPetRequest) returns (GetPetResponse);
}
```

The package and namespace are lower/pascal versions of the service name because no explicit values were given. `petId` became `pet_id`; `limit` and `page` kept their names. `ListPets` is the only RPC with two request fields, so it is the only one without a scalar convenience overload.

The CLI reports the same document in one screen:

TEXT

```
spec:       docs/manual/fixtures/petstore.yaml
service:    PetstoreService
namespace:  PetstoreService
package:    petstoreservice
rpc:        PetstoreService (3)
  ListPets - GET /pets (Unary)
  CreatePet - POST /pets (Unary)
  GetPet - GET /pets/{petId} (Unary)
messages:   7
enums:      1
```

Seven messages: `Pet`, three request wrappers, three response wrappers. One enum. Had `listPets` declared `page` and `limit` but no `next_page_token`, it would still be unary; pagination detection only changes the kind when a cursor or next-page-token parameter is present (chapter 13).

The implementation and the call

```C#
internal sealed class PetServiceImpl : PetstoreServiceBase
{
    private readonly Dictionary<long, Pet> _pets = new();

    public override Task<GetPetResponse> GetPetAsync(GetPetRequest request, ServerCallContext context)
    {
        if (!_pets.TryGetValue(request.PetId, out var pet))
            throw new RpcException(new Status(StatusCode.NotFound, $"pet {request.PetId} not found"));
        return Task.FromResult(new GetPetResponse { Data = pet });
    }

    public override Task<CreatePetResponse> CreatePetAsync(CreatePetRequest request, ServerCallContext context)
    {
        _pets[request.Body.Id] = request.Body;
        return Task.FromResult(new CreatePetResponse { Data = request.Body });
    }

    public override Task<ListPetsResponse> ListPetsAsync(ListPetsRequest request, ServerCallContext context)
    {
        var response = new ListPetsResponse();
        response.Items.Add(_pets.Values.OrderBy(p => p.Id).Skip((request.Page - 1) * request.Limit)
            .Take(request.Limit));
        return Task.FromResult(response);
    }
}
```

`request.Body` is non-null by convention, not by validation: a client can send `CreatePetRequest` with no body, and the handler must check it if that matters. `Data` is a nullable message field; a response that leaves it null is legal protobuf and a consumer must handle it.

The client side is where the mapping becomes ergonomic:

```C#
using var channel = GrpcChannel.ForAddress("http://localhost:50051");
var client = PetstoreServiceClient.Create(channel);

var created = await client.CreatePetAsync(new CreatePetRequest
{
    Body = new Pet { Id = 3, Name = "Luna", Status = StatusEnum.Sold }
});

var fetched = await client.GetPetAsync(3);            // long → GetPetResponse
var name = (await client.GetPetDataAsync(3)).Name;    // long → Pet
var pets = await client.ListPetsAsync(new ListPetsRequest { Limit = 20, Page = 1 });
```

`GetPetAsync(3)` exists because `GetPetRequest` has one field; `GetPetDataAsync(3)` exists because `GetPetResponse` has one field too. `ListPetsAsync` has no scalar overload because its request has two fields, and its response field `Items` is repeated, so a `ListPetsItemsAsync` overload would hand back the whole `RepeatedField<Pet>` rather than an element.

What this example does not exercise

It has no `oneOf`, no `Struct`, no binary body, no pagination token, no security scheme and no proxy. Each of those changes one part of the surface and is covered where it belongs: chapters 9, 15, 16, 13, 21 and 22. The value of starting here is that the ordinary path is fully specified, so a surprise later can be attributed to the construct that caused it rather than to the mapping as a whole.

29. Glossary

Terms as this manual uses them. Where a term has a narrower meaning here than in the wider ecosystem, the definition says so.

AdditionalFiles — the MSBuild item group a source generator reads. One entry per OpenAPI document, with metadata for the service name and options (chapter 4).

AOT (NativeAOT) — publishing a .NET program as a native executable with no runtime code generation. The generated messages implement serialization by hand so the result needs no reflection (chapters 1 and 18).

allOf — JSON Schema composition meaning "all of these". Merged into one message, in order, with duplicate properties kept (chapter 9).

base class — `{Service}Base`, the abstract class a server implements. It carries `[BindServiceMethod]` and one virtual member per RPC (chapter 19).

BindService — the generated static method that binds a base-class instance to a server: one overload returns a `ServerServiceDefinition`, the other registers the implementation through a `ServiceBinderBase` (chapter 19).

body message — the request message's field that carries the HTTP request body. Named `Body`; its type is the body schema, or a generated `<Method>Body` wrapper (chapter 14).

channel — `GrpcChannel`, the client-side connection and HTTP/2 transport. The generated client does not create one; the application does (chapter 20).

client-streaming — an RPC where the client sends a stream and the server answers once. Produced by `x-grpc-streaming: client` or the large-POST heuristic (chapter 13).

contract class — `{Service}`, the static class holding `ServiceFullName`, the `Method_*` descriptors, the marshallers, `BindService` and the nested low-level client (chapter 19).

descriptor — here, the `.proto` text in `{Service}Proto.Text`, not a `FileDescriptor` object. The generated messages return `null` from `Descriptor` (chapters 3 and 18).

DG (dependency injection) — the generated `AddSharpPortico{Service}()` extension registers the convenience client from an existing `GrpcChannel`; nothing else (chapters 19 and 27).

enum — a protobuf enumeration generated from a JSON Schema `enum`. Members are sanitised to PascalCase, and the first member must be zero for `protoc` to accept the descriptor (chapter 11).

grpc-dotnet — the ASP.NET Core gRPC host (`AddGrpc`, `MapGrpcService`). It uses the `ServiceBinderBase` overload of `BindService` (chapter 19).

Grpc.Core — the native gRPC library (`Server`, `Channel`, `ClientBase`). The generated contract and nested client target it directly (chapters 19 and 20).

hint name — the file stem used for the descriptor class and the generated file name. It comes from the `AdditionalFiles` path, not from `info.title` (chapter 4).

IMessage<T> — the Google.Protobuf interface with `MergeFrom`, `WriteTo`, `CalculateSize` and a `Descriptor`. The generated messages implement it without a base class (chapter 18).

mapping — the whole transform from OpenAPI to the intermediate model. Not a synonym for serialization; the gateway's JSON mapping is a separate, runtime concern (chapters 7–17 and 23).

optional — the proto3 label that gives a scalar explicit presence. Emitted for a nullable scalar, with `HasX` / `ClearX` members (chapter 12).

package — the protobuf package in the descriptor, derived from the service name or set by `ProtoPackage`. It is part of the wire identity (chapter 8).

pagination — detection of `page` / `limit` / `cursor` / `next_page_token` parameters by name. A cursor or token turns the RPC into server-streaming; page/limit alone does not (chapter 13).

presence — whether a field was set, distinct from its value. Only `optional` scalars, message fields and `bytes` can express it (chapter 12).

proxy — `{Service}Proxy`, the generated `{Service}Base` that answers gRPC by calling the legacy REST service. Unary only (chapters 22 and 23).

repeated — protobuf's sequence type, generated for a JSON array. It has no null and no per-element presence (chapter 10).

request wrapper — `{Method}Request`, the message an RPC accepts. Always generated, even when it has only `has_value` (chapters 13 and 14).

response wrapper — `{Method}Response`, the message an RPC returns. Its success payload is `Data`, `Items`, or `Body`, depending on the shape (chapter 13).

RPC kind — unary, server-streaming, client-streaming or bidi-streaming. Chosen by the `x-grpc-streaming` extension, the large-POST heuristic, or pagination (chapter 13).

schema mapper — `SchemaMapper`, the component that turns an OpenAPI schema into a message, enum or field model. It owns types, shapes and presence (chapters 7–12).

service — one generated gRPC service per document. The document's `tags` do not split it (chapter 13).

source generator — the Roslyn incremental generator that runs during compilation and writes the C# files. It is not a CLI and does not invoke `protoc` (chapter 1).

Struct — `google.protobuf.Struct`, the well-known type for "any JSON object". Used for a free-form object in a field position; it has no proxy serializer (chapters 9, 15 and 23).

unary — one request, one response. The default kind and the only kind the proxy overrides (chapters 13 and 22).

Value — `google.protobuf.Value`, the well-known type for any JSON value. Used for multi-type unions and tuples; like `Struct`, it has no proxy serializer (chapters 6 and 23).

well-known types — protobuf's predefined messages (`Struct`, `Value`, `Timestamp`, `Duration`, `Any`, `Empty`, wrappers). The descriptor imports the ones the model uses (chapter 15).

wire tag — the field number and wire type byte that precede a value on the wire. The generator emits it as a `const uint Tag_N_Name` per field (chapter 18).

x-grpc-streaming — the operation-level extension that selects an RPC kind. Values are case-sensitive: `unary`, `client`, `server`, `bidi`; anything else is unary (chapter 13).